

ચાલો, હવે આપણે જાવાસ્ક્રિપ્ટ અને નોડ.જીએસના સૌથી શક્તિશાળી કન્સેપ્ટ એટલે કે **Macrotask Queue** અને **Microtask Queue** ના અસલી આંતરિક એન્જિનિયરિંગમાં એકદમ **Deep (ઊંડાણપૂર્વક)** આગળ વધીએ.

જ્યારે ઇવેન્ટ લૂપ (Event Loop) ચાલતું હોય, ત્યારે કમ્પ્યુટરની રેમ (RAM) માં કયા કોડને પહેલી પ્રાયોરિટી મળે છે અને કયો કોડ લાઇન પૂરી થયા પછી રન થાય છે, તે આપણે કોઈ પણ ઇનલાઇન મિક્સિંગ વગર, તદ્દન નવી લાઇનમાં (Line by Line) સાદી ગુજરાતી ભાષામાં સમજીએ.

■ JAVASCRIPT MASTER RADAR: RUNTIME CONCURRENCY ENGINE

૫૪. Macrotask Queue (મેક્રોટાસ્ક ક્યુ — સાદી ઓપરેશન લાઇન)

🔴 વ્યાખ્યા:

Macrotask Queue (જેને ઘણીવાર ફક્ત Task Queue પણ કહેવાય છે) એ એક એવી લાઇન છે જેમાં બ્રાઉઝર અથવા ઓપરેટિંગ સિસ્ટમ દ્વારા સોંપેલા મોટા અને સામાન્ય અસિન્ક્રોનસ કામોના કોલબેક્સ આવીને જમા થાય છે.

⚙️ આંતરિક એન્જિનિયરિંગ:

- **સભ્યો (Components):** આ લાઇનની અંદર `setTimeout()`, `setInterval()`, સાદા DOM ઇવેન્ટ લિસનર્સ (જેમ કે click), અને નોડ.જીએસનું `setImmediate()` આવીને ઊભા રહે છે.
- **નિયમ:** ઇવેન્ટ લૂપ પોતાના એક ચક્ર (Tick) માં આ મેક્રોટાસ્ક ક્યુમાંથી **માત્ર ૧ જ સિંગલ ટાસ્ક** ને ઉપાડે છે, તેને Call Stack પર મોકલીને રન કરે છે, અને પછી તુરંત જ બીજી બધી લાઇન ચેક કરવા આગળ વધી જાય છે.

૫૫. Microtask Queue (માઇક્રોટાસ્ક ક્યુ — હાઇ-પ્રાયોરિટી લાઇન)

🔴 વ્યાખ્યા:

Microtask Queue એ એક અત્યંત શક્તિશાળી અને વીઆઇપી (VIP) લાઇન છે, જેનો પાવર સાદી મેક્રોટાસ્ક ક્યુ કરતાં કરોડો ગણો વધારે હોય છે.

⚙️ આંતરિક એન્જિનિયરિંગ:

- **સભ્યો (Components):** આ લાઇનની અંદર જાવાસ્ક્રિપ્ટના બધા જ **Promises** (`.then()`, `.catch()`, `.finally()`), `async/await` નું ઇન્ટરનલ પ્રોસેસિંગ, અને બ્રાઝરનું **MutationObserver** આવીને લાઇન લગાવે છે.
- **Priority Hijacking (લાઇન હાઇજેક કરવાનો નિયમ):** ઇવેન્ટ લૂપ જ્યારે પણ પોતાનો કોઈ ચાલુ ફેઝ પૂરો કરીને આગલા સ્ટેપ પર જવાનો ટ્રાય કરશે, તેની બરાબર વચ્ચે તે આ માઇક્રોટાસ્ક ક્યુ પાસે આવીને ઊભું રહેશે.
- તે નિયમ પાળે છે કે: **"જ્યાં સુધી આ માઇક્રોટાસ્ક ક્યુની અંદર બનેલા તમે તમામ (All) વીઆઇપી કામો ૧૦૦% પૂરા થઈને આખી લાઇન ખાલી (Empty) ન થઈ જાય, ત્યાં સુધી ઇવેન્ટ લૂપ આગળના કોઈ પણ મેક્રોટાસ્ક (setTimeout) ને અડશે પણ નહીં!"**

૫૬. process.nextTick() (નેક્સ્ટ-ટીક — સુપર કમાન્ડર લાઇન)

🔴 વ્યાખ્યા:

`process.nextTick()` એ ખાસ કરીને **Node.js** ની અંદર અસિન્ક્રોનસ કામોને સુપર પ્રાયોરિટી આપવા માટેનું એક અલ્ટીમેટ મેકેનિઝમ છે.

⚙️ આંતરિક એન્જિનિયરિંગ:

- **સર્વોચ્ચ સત્તા (The Highest Priority):** આ આખા જાવાસ્ક્રિપ્ટ રન-ટાઇમનો સૌથી મોટો કમાન્ડર છે. આની ક્યુ માઇક્રોટાસ્ક ક્યુ કરતાં પણ પહેલાં પ્રોસેસ થાય છે!
- જો તમે કોડમાં આ કમાન્ડ લખશો, તો ઇવેન્ટ લૂપ અત્યારે જે પણ કામ કરી રહ્યું છે, તે કરન્ટ ઓપરેશન પૂરું થતાં જ (ફેઝ બદલાય તેની બરાબર પહેલાં) આને રન કરશે.
- **🔥 Infinite Loop Trap:** જો તમે ભૂલથી એક `process.nextTick` ની અંદર બીજો `nextTick` વારંવાર કોલ કર્યા કરશો, તો ઇવેન્ટ લૂપ આખી જિંદગી એ જ સફાઈ કરવામાં અટકી જશે, તે ક્યારેય આગળના કોઈ ફેઝ કે `setTimeout` પર જઈ જ નહીં શકે, જેનાથી આખું સર્વર એક સેકન્ડમાં જામ (**Starve the Event Loop**) થઈ જશે!

📺 ૫૭. સંપૂર્ણ એડવાન્સ મેક્રો-માઇક્રોટાસ્ક પ્રાયોરિટી એન્જિનિયરિંગ પ્રોજેક્ટ કોડ

આપણે આખું એડવાન્સ **Macrotask**, **Microtask**, અને `process.nextTick` ના ટાઈમિંગ વહીવટને ૧૦૦% સાબિત કરતો એક અદ્ભુત ઇન્ડસ્ટ્રી ગ્રેડ ટેસ્ટ કોડ નીચે મુજબ સેટ કરીએ:

javascript

"use strict";

// ૧. સાદો સિંક્રોનસ કોડ (સૌથી પહેલાં Call Stack માં ડાયરેક્ટ રન થશે)

console.log("🌀 ૧. START: મેઇન સિંક્રોનસ એન્જિન ઓન થયું.");

// ૨. MACROTASK: આ ટાઈમર Macrotask Queue ની અંદર લાઇન લગાવશે

setTimeout(() => {

 console.log("🕒 ૨. setTimeout: (મેક્રોટાસ્ક — આ બધાયની પાછળ રન થશે)");

}, 0);

// ૩. MICROTASK: આ પ્રોમિસ Microtask Queue ની અંદર લાઇન લગાવશે

Promise.resolve().then(() => {

 console.log("💎 ૩. PROMISE: (માઇક્રોટાસ્ક — આ સાદા ટાઈમર પહેલાં આખી લાઇન હાઇજેક કરશે)");

});

// ૪. SUPER MICROTASK: આ સુપર કમાન્ડ છે

```
process.nextTick(() => {
```

```
  console.log(" ⚡ ૪. process.nextTick: (સુપર પ્રાયોરિટી — આ પ્રોમિસ કરતાં પણ પહેલાં રન થશે!);
```

```
});
```

// ૫. છેલ્લો સિંક્રોનસ કોડ

```
console.log(" 🟡 ૫. END: મેઇન સિંક્રોનસ એન્જિન પ્રોસેસ પૂરી થઈ.");
```

Use code with caution.

🧠 આ એડવાન્સ કોડનો અસલી આઉટપુટ ક્રમ (The Hardcore Execution Order):

જ્યારે તમે આ કોડ લાઈવ રન કરશો, ત્યારે સ્ક્રીન પર આ ચોક્કસ કમાન્ડ સિક્વન્સમાં જ આઉટપુટ પ્રિન્ટ થશે, જે લૂપના આંતરિક વાયરિંગને સાબિત કરે છે:

1. 🏠 ૧. START (Call Stack - સિંક્રોનસ ફર્સ્ટ)
2. 🟡 ૫. END (Call Stack - સિંક્રોનસ સેકન્ડ)
3. ⚡ ૪. process.nextTick (સુપર કમાન્ડ - ચાલુ લાઇન પૂરી થતાં જ પહેલો મોકો મળ્યો)
4. 💧 ૩. PROMISE (માઇક્રોટાસ્ક ક્યુ - સાદા ટાસ્ક કરતાં પહેલાં કમ્પ્લીટ થઈ)
5. ⌚ ૨. setTimeout (મેક્રોટાસ્ક ક્યુ - છેલ્લે આખી સિસ્ટમ ખાલી થતાં વારો આવ્યો)

અહીં આપણો આખો **Macrotask, Microtask અને Concurrency Engine** નો આખો PhD આર્કિટેક્ચર કન્સેપ્ટ ૧૦૦% વિગતવાર પૂરો થાય છે!

📌 PART 1: THE JAVASCRIPT LANGUAGE FOUNDATIONS (ભાષાનો પાયો)

૧. JavaScript Engines (જાવાસ્ક્રિપ્ટ એન્જિન્સ)

- **વિગતવાર સમજૂતી:** ગૂગલ ક્રોમનું V8, ફાયરફોક્સનું SpiderMonkey અને સફારીનું JavaScriptCore એવા સોફ્ટવેર પ્રોગ્રામ્સ છે જે બ્રાઉઝરમાં જાવાસ્ક્રિપ્ટ કોડને કમ્પ્યુટરના પ્રોસેસર સમજી શકે તેવા મશીન કોડ (0 અને 1) માં ફેરવે છે.

૨. Just-In-Time (JIT) Compilation (જસ્ટ-ઇન-ટાઇમ કમ્પાઇલેશન)

- **વિગતવાર સમજૂતી:** આ એક આધુનિક ટેકનોલોજી છે જે કોડ રન થતો હોય તેની સાથે જ (On-the-fly) તેને ઓપ્ટિમાઇઝ્ડ મશીન કોડમાં કમ્પાઇલ કરે છે, જેનાથી જાવાસ્ક્રિપ્ટની સ્પીડ ૧૦૦ ગણી વધી જાય છે.

૩. Ignition Interpreter (ઇગ્નિશન ઇન્ટરપ્રિટર)

- **વિગતવાર સમજૂતી:** V8 એન્જિનનો આ પ્રથમ ગેટ છે. તે કોડ લોડ થતાં જ કમ્પાઇલેશનની રાહ જોયા વગર તેને તાત્કાલિક **Bytecode** માં ફેરવીને પ્રોગ્રામને સેકન્ડના હજારમા ભાગમાં ચાલુ કરી દે છે.

૪. TurboFan Optimizer (ટર્બોફેન ઓપ્ટિમાઇઝર)

- **વિગતવાર સમજૂતી:** V8 નો આ સુપર-કમ્પાઇલર ભાગ છે. પ્રોગ્રામ ચાલુ હોય ત્યારે જે ફંક્શન વારંવાર રન થતું હોય (Hot Code), તેને આ મશીન સીધું જ હાર્ડકોર મશીન કોડમાં કન્વર્ટ કરીને પ્રોસેસર પર રોકેટ સ્પીડે રન કરે છે.

૫. Abstract Syntax Tree (AST) (એબ્સ્ટ્રેક્ટ સિન્ટેક્સ ટ્રી)

- **વિગતવાર સમજૂતી:** V8 એન્જિન જ્યારે તમારો કોડ વાંચે છે, ત્યારે તે તેના લખાણને તોડીને મેમરીમાં એક વૃક્ષ જેવું બાઇનરી માળખું બનાવે છે, જેને AST કહેવાય છે, આના આધારે જ આગળ બાઇટકોડ બને છે.

૬. Code Deoptimization (ડી-ઓપ્ટિમાઇઝેશન બગ)

- **વિગતવાર સમજૂતી:** જો ટર્બોફેન કોઈ ફંક્શનને નંબર ડેટા ટાઇપ સમજીને ઓપ્ટિમાઇઝ કર્યું હોય, અને અચાનક યુઝર તેમાં સ્ટ્રિંગ પાસ કરી દે, તો એન્જિન કન્ફ્યુઝ થઈને ઓપ્ટિમાઇઝ કોડ ફેંકી દે છે અને પાછો સ્વો મોડ પર આવી જાય છે.

૭. Execution Context (એક્ઝિક્યુશન કન્ટેક્સ્ટ)

- **વિગતવાર સમજૂતી:** જાવાસ્ક્રિપ્ટની મેમરીમાં બનતું એક એવું વર્ચ્યુઅલ બોક્સ છે જેની અંદર અત્યારે કોડના કયા વેરીએબલ્સ લોડ છે, કયું ફંક્શન ચાલુ છે અને કયો સ્કોપ એક્ટિવ છે તેનો આખો લાઇવ નકશો સચવાય છે.

૮. Global Scope (ગ્લોબલ સીમાડો)

- **વિગતવાર સમજૂતી:** કોઈપણ ફંક્શન કે ક્લોઝર્ બ્રેકેટ્સ { } ની બહાર બનેલો વેરીએબલ ગ્લોબલ સ્કોપમાં હોય છે. આ વેરીએબલ આખા પ્રોજેક્ટની ફાઇલમાં ગમે ત્યાંથી એક્સેસ અને મોડિફાઇ કરી શકાય છે.

૯. Script Scope (સ્ક્રીપ્ટ સ્કોપ)

- **વિગતવાર સમજૂતી:** ES6 નો ગુપ્ત નિયમ. ગ્લોબલ એરિયામાં બનેલા let અને const વેરીએબલ્સ આ ડબ્બામાં સેવ થાય છે, જેથી તે આખા પેજ પરથી એક્સેસ તો થાય છે પણ ગ્લોબલ window ઓબ્જેક્ટની અંદર કોપી નથી થતા, જે હેકિંગ પ્રોટેક્શન આપે છે.

૧૦. Local / Function Scope (લોકલ સ્કોપ)

- **વિગતવાર સમજૂતી:** ફંક્શનની અંદર બનેલા વેરીએબલ્સ ફંક્શનની પોતાની પ્રાઇવેટ જેલમાં લોક હોય છે. ફંક્શનની બહાર આખા પ્રોગ્રામમાં તેને કોઈ જોઈ કે અડી શકતું નથી.

૧૧. Block Scope (બ્લોક સ્કોપ)

- **વિગતવાર સમજૂતી:** સાદા ક્લોઝર્ બ્રેકેટ્સ { } (જેમ કે if કે for લૂપ) નો સ્કોપ. માત્ર let અને const જ આ નિયમ પાળે છે અને બ્લોક પૂરો થતાં જ રેમમાંથી ઓટોમેટિક નાશ પામે છે.

૧૨. Scope Pollution (સ્કોપ પોલ્યુશન બગ)

- **વિગતવાર સમજૂતી:** જો તમે કોઈ બ્લોકની અંદર ભૂલથી જૂનો var વેરીએબલ વાપરશો, તો તે બ્લોકની દીવાલો તોડીને બહાર નીકળી ગ્લોબલ વિન્ડો સાથે ચોંટી જશે અને મેમરી બગાડશે, જેને સ્કોપ પોલ્યુશન કહેવાય છે.

૧૩. Variable & Function Hoisting (હોઇસ્ટિંગ મિકેનિઝમ)

- **વિગતવાર સમજૂતી:** કોડ રન થાય તે પહેલાં V8 એન્જિન બધા વેરીએબલ્સ અને ફંક્શન ડિક્લેરેશન્સને મેમરીમાં સૌથી ઉપર (Top) મોકલી દે છે. var હોઇસ્ટ થતાં undefined બને છે, જ્યારે ફંક્શન બન્યા પહેલાં પણ રન થઈ શકે છે.

૧૪. Temporal Dead Zone (TDZ) (ટેમ્પોરલ ડેડ ઝોન)

- **વિગતવાર સમજૂતી:** let અને const હોઇસ્ટ થયા પછી જ્યાં સુધી કોડમાં પોતાની અસલી લાઇન પર ન પહોંચે, ત્યાં સુધી તે મેમરીમાં એક ગુપ્ત પ્રતિબંધિત એરિયામાં લોક રહે છે. આ ઝોનમાં તેને એક્સેસ કરતી જ ReferenceError આવે છે.

૧૫. Type Coercion (Implicit vs Explicit)

- **વિગતવાર સમજૂતી:** Implicit એટલે જાવાસ્ક્રિપ્ટ એન્જિન પોતાની મેળે ઓટોમેટિક ડેટા ટાઇપ બદલે (દા.ત. 5 + "5" = "55"). Explicit એટલે ડેવલપર પોતે જાણીજોઈને ફંક્શન વાપરીને ટાઇપ બદલે (દા.ત. Number("10")).

૧૬. Loose Equality (==) vs Strict Equality (===)

- **વિગતવાર સમજૂતી:** Double Equals (==) ડેટા ટાઇપ જોયા વગર ફક્ત અંદરની કિંમત સરખાવે છે. જ્યારે Triple Equals (===) કિંમત અને ડેટા ટાઇપ બંને ૧૦૦% સેમ હોવા પર જ સાચું (true) આપે છે, જે ઇન્ડસ્ટ્રી સ્ટાન્ડર્ડ છે.

૧૭. Short-Circuit Evaluation (લોજિકલ શોર્ટ-સર્કિટ)

- **વિગતવાર સમજૂતી:** && ઓપરેટરને પહેલી જ શરત જો false મળી જાય, અથવા || ઓપરેટરને પહેલી શરત true મળી જાય, તો તે સમય બગાડ્યા વગર આગળનો કોડ ચેક કર્યા વગર ત્યાંથી જ આખું રિઝલ્ટ આપીને બહાર નીકળી જાય છે.

PART 2: OBJECTS & MEMORY MANAGEMENT (ઓબ્જેક્ટ્સ અને મેમરી)

૧૮. Call Stack Memory (કોલ સ્ટેક મેમરી)

- **વિગતવાર સમજૂતી:** V8 એન્જિનની અંદર રહેલું એક સુપર-ફાસ્ટ, ફિક્સ સાઇઝનું માળખું જે LIFO (Last In, First Out) નિયમ પર કામ કરે છે. તે પ્રોગ્રામના ફંક્શન ઓર્ડર અને સાદા પ્રિમિટિવ વેરીએબલ્સને સાચવે છે.

૧૯. Heap Allocation Memory (હીપ મેમરી)

- **વિગતવાર સમજૂતી:** રેમ (RAM) ની અંદર આવેલી એક બહુ જ મોટી, અનસ્ટ્રક્ચર્ડ જગ્યા જ્યાં બધા જ જટિલ અને મોટા ડેટા ઓબ્જેક્ટ્સ (Objects, Arrays) નો અસલી ભારે ભારે ડેટા સ્ટોર થાય છે.

૨૦. Memory Address Reference (મેમરી પોઇન્ટર)

- **વિગતવાર સમજૂતી:** ઓબ્જેક્ટનો અસલી ડેટા Heap માં સેવ થાય છે, પણ એ જગ્યાનું જે સરનામું (Address Pointer) હોય છે, તે સાઇઝમાં નાનું હોવાથી આપણો Call Stack તેને પોતાની ફ્રેમમાં કોપી તરીકે સાચવી રાખે છે.

૨૧. Value vs Reference Types

- **વિગતવાર સમજૂતી:** Primitives (Number, String) એ Value Types છે જેની નવી સ્વતંત્ર કોપી બને છે. Objects અને Arrays એ Reference Types છે જેમાં અસલી ડેટા કોપી નથી થતો, ફક્ત મેમરી સરનામું શેર થાય છે.

૨૨. Shallow Copy (Spread Operator ...)

- **વિગતવાર સમજૂતી:** આ ઓબ્જેક્ટના પહેલા વેયરને તો નવી મેમરીમાં સેફ કોપી કરી લે છે, પણ જો ઓબ્જેક્ટની અંદર પણ બીજો કોઈ નેસ્ટેડ (Nested) ઓબ્જેક્ટ હોય, તો તેનું અંદરનું મેમરી સરનામું જૂનું જ રહે છે.

૨૩. Deep Copy via structuredClone()

- **વિગતવાર સમજૂતી:** ઓરિજિનલ ઓબ્જેક્ટ સાથેથી મેમરીના તમામ વાયરો કાયમ માટે તોડીને, ગમે તેટલા કરોડો વેયર ઊંડા નેસ્ટેડ ડેટાની પણ ૧૦૦% બ્રાન્ડ-ન્યુ કોપી બનાવવાની સત્તાવાર આધુનિક મેથડ.

૨૪. V8 Garbage Collection (ગાર્બેજ કલેક્શન)

- **વિગતવાર સમજૂતી:** બેકગ્રાઉન્ડમાં ચાલતો એક ઓટોમેટિક પ્રોગ્રામ જે મેમરી (Heap) માં એવા ઓબ્જેક્ટ્સને શોધીને રેમમાંથી કાયમ માટે ડીલીટ કરી નાખે છે જેનો પ્રોગ્રામમાં કોઈ પણ જગ્યાએ વાયર કે રેફરન્સ બાકી રહ્યો નથી.

૨૫. Scavenger GC Phase (Young Generation)

- **વિગતવાર સમજૂતી:** ન્યૂ સ્પેસ (New Space) ને સાફ કરવાનો હાઇ-સ્પીડ અલ્ગોરિથમ. તે **From-Space** માંથી જીવતા ઓબ્જેક્ટ્સ ફાસ્ટ સ્કેન કરીને બાજુના ખાલી **To-Space** માં કોપી કરી લે છે અને બાકીનો કચરો એક ઝાટકે સાફ કરે છે.

૨૬. Mark-Sweep GC Phase (Old Generation)

- **વિગતવાર સમજૂતી:** ઓલ્ડ સ્પેસ (Old Space) ને સાફ કરવાનો મોટો પ્રોગ્રામ. તે ગ્લોબલ રુટ્સથી મુસાફરી શરૂ કરીને જીવતા ઓબ્જેક્ટ પર **Mark** (નિશાન) લગાવે છે અને બાકીના અન-રેફરન્સ કચરાને **Sweep** (Delete) કરી મેમરી મુક્ત કરે છે.

૨૭. Memory Leak Vectors (મેમરી લીક થવાના જોખમો)

- **વિગતવાર સમજૂતી:** જ્યારે કોઈ ઓબ્જેક્ટનું કામ પૂરું થઈ જાય, છતાં અજાણતા તેનો કોઈ નાનો વાયર ગ્લોબલ સ્કોપ કે ટાઇમર સાથે જોડાયેલો રહી જાય, ત્યારે ગાર્બેજ કલેક્ટર તેને ક્યારેય ડીલીટ નથી કરી શકતું, જેનાથી રેમ ભરાતી જાય છે અને સર્વર કેશ થાય છે.

૨૮. Object.freeze() (ઓબ્જેક્ટ ફ્રીઝ પ્રોટોકોલ)

- **વિગતવાર સમજૂતી:** ઓબ્જેક્ટની બધી કી ના ઇન્ટરનલ ઓબ્જેક્ટ ડિસ્ક્રિપ્ટર્સ writable: false અને configurable: false કરી દે છે, જેથી પ્રોગ્રામમાં કોઈ નવી કી ઉમેરી શકાતી નથી, જૂની ડીલીટ નથી કરી શકાતી કે તેની અંદરની વેલ્યુ પણ બદલી શકાતી નથી.

૨૯. Object.seal() (ઓબ્જેક્ટ સીલ પ્રોટોકોલ)

- **વિગતવાર સમજૂતી:** આ ઓબ્જેક્ટની બધી પ્રોપર્ટીઝના configurable: false કરી દે છે પણ writable ને true રાખે છે. આનાથી ઓબ્જેક્ટમાં નવી કી પ્લસ કે માઇનસ નથી કરી શકાતી, પણ જે જૂની કી હાજર છે તેની અંદરનો વેલ્યુ ડેટા બદલી શકાય છે.

30. Object Property Descriptors (પ્રોપર્ટી ડિસ્ક્રિપ્ટર્સ)

- **વિગતવાર સમજૂતી:** ઓબ્જેક્ટની દરેક કી ની પાછળ છુપાયેલા 3 સિક્યોરિટી વેલ્યુ લોક—writable (કિંમત બદલવી કે નહીં), enumerable (for...in લૂપમાં દેખાડવી કે હાઇડ રાખવી), અને configurable (ડિલીટ કરવા દેવી કે લોક રાખવી).

31. JavaScript Proxies & Reflect API (પ્રોક્સી પ્રોટેક્શન શીલ્ડ)

- **વિગતવાર સમજૂતી:** ઓરિજિનલ ઓબ્જેક્ટની આજુબાજુ બનેલું એક પાવરફુલ સુરક્ષા કવચ. બહારનો કોઈ કોડ ઓબ્જેક્ટને સીધો અડી શકતો નથી, વચ્ચે રહેલા **Get/Set Traps** તેને હવામાં પકડીને સિક્યોરિટી ચેક કરીને જ ડેટા બદલવા કે જોવા દે છે.

🌀 PART 3: ADVANCED RUNTIME CONCURRENCY (ઇવેન્ટ લૂપ અને અસિંક)

32. The Event Loop Mechanics (ઇવેન્ટ લૂપનું ચક્ર)

- **વિગતવાર સમજૂતી:** libuv C++ લાઇબ્રેરી દ્વારા ચાલતું એક એવું મેકેનિઝમ જે સિંગલ-થ્રેડેડ જાવાસ્ક્રિપ્ટને Non-blocking પાવર આપે છે. તે Call Stack ખાલી થતાં જ જાગે છે અને અસિંક્રોનસ કામોના કોલબેકસને લાઇનસર પ્રોસેસ કરે છે.

33. The 6 Loop Phases (ઇવેન્ટ લૂપના ૬ ચક્રો)

- **વિગતવાર સમજૂતી:** લૂપનો દરેક રાઉન્ડ (Tick) આ ૬ ફેઝમાંથી પસાર થાય છે: Timers (setTimeout) -> Pending OS Callbacks -> Idle/Prepare (સિસ્ટમ ઇન્ટરનલ) -> Poll (નવો નેટવર્ક ડેટા બ્લોકિંગ) -> Check (setImmediate) -> Close Callbacks (કનેક્શન ક્લીનઅપ).

34. Microtask Priority Hijacking (માઇક્રોટાસ્ક વીઆઇપી ક્યુ)

- **વિગતવાર સમજૂતી:** આ ઇવેન્ટ લૂપની સૌથી શક્તિશાળી VIP લાઇન છે જેમાં બધા **Promises (.then/.catch)** અને **async/await** ના કોડ ઊભા રહે છે. લૂપ ગમે તે ફેઝમાં હોય, પણ જ્યાં સુધી આ માઇક્રોટાસ્ક ક્યુ પૂરેપૂરી ખાલી ન થાય, ત્યાં સુધી તે આગળ વધી શકતું નથી.

35. process.nextTick() Scheduling (સુપર કમાન્ડર)

- **વિગતવાર સમજૂતી:** Node.js નો આ સર્વોચ્ચ કમાન્ડર છે જેનો પાવર પ્રોમિસ કરતાં પણ પહેલો છે. ઇવેન્ટ લૂપ પોતાનો ચાલુ ફેઝ પૂરો કરીને આગલા સ્ટેપ પર પગ મૂકે તેની બરાબર વચ્ચે આ nextTick નો કોડ સૌથી પહેલા રન કરવો જ પડે છે.

36. Stack Overflow Vectors (મેક્સિમમ કોલ સ્ટેક કેશ)

- **વિગતવાર સમજૂતી:** જ્યારે કોઈ ફંક્શન પોતાની અંદર જ પોતાને કોઈ પણ અટકવાના નિયમ (Base Case) વગર વારંવાર કોલ કર્યા કરે (Infinite Recursion), ત્યારે Call Stack પૂરેપૂરો ઉપર સુધી ભરાઈ જાય છે અને એન્જિન Maximum call stack size exceeded ની એરર આપીને કેશ થઈ જાય છે.

37. Async / Await Generator Pause Engine

- **વિગતવાર સમજૂતી:** જ્યારે કોડમાં await આવે, ત્યારે V8 એન્જિન આખું સર્વર જામ નથી કરતું. તે બેકગ્રાઉન્ડમાં ફંક્શનના એક્ઝિક્યુશનને ત્યાં જ **Pause** (અટકાવી) દે છે અને મેઇન થ્રેડ મુક્ત રાખે છે. જવાબ આવતા જ માઇક્રોટાસ્ક ક્યુમાંથી કોડ પાછો ત્યાંથી જ **Resume** થાય છે.

⚙️ PART 4: COMPILATION, CLOSURES & OOP (આર્કિટેક્ચર નિયમો)

3૮. `[[Scopes]]` Closure Memory Object

- **વિગતવાર સમજૂતી:** જ્યારે પેરેન્ટ ફંક્શન રન થઈને કોલ સ્ટેકમાંથી નાશ પામે, છતાં યાદ્દત ફંક્શન તેના વેરીએબલ્સ વાપરતું હોય, ત્યારે V8 એન્જિન એ વેરીએબલ્સને સ્ટેકમાંથી ઉઠાવીને Heap મેમરીમાં એક ખાસ પ્રાઇવેટ ઓબ્જેક્ટમાં લોક કરી દે છે, જેને `[[Scopes]]` Closure કહેવાય છે.

3૯. Data Encapsulation (ડેટા કેપ્સ્યુલેશન)

- **વિગતવાર સમજૂતી:** ઇમ્પોર્ટન્ટ વેરીએબલ્સને ફંક્શનની અંદર છુપાવીને (Private કરીને) બહારની દુનિયાના કોડથી પ્રોટેક્ટ કરવા. ડેટા બદલવા કે જોવા માટે આપણે માત્ર કસ્ટમ સુરક્ષિત ગેટ્સ (Getters and Setters) જ ઓપન રાખીએ છીએ.

૪૦. Factory Functions Design Pattern

- **વિગતવાર સમજૂતી:** સોફ્ટવેર આર્કિટેક્ચરની એવી સુંદર પેટર્ન જેમાં કોઈ class કે new કીવર્ડ વગર, સાદા ફંક્શનની અંદર ક્લોઝર સેટ કરીને ઓટોમેટિક એક સિક્યોર કસ્ટમ ઓબ્જેક્ટ મેન્યુફેક્ચર (બનાવીને) બહાર મોકલવામાં આવે છે.

૪૧. Function Currying & Infinite Chains

- **વિગતવાર સમજૂતી:** એકસાથે બધી આર્ગ્યુમેન્ટ્સ લેવાના બદલે ફંક્શનને એવા ટુકડાઓમાં ફેરવવું જ્યાં દરેક ફંક્શન માત્ર ૧ જ આર્ગ્યુમેન્ટ લે અને અંદરથી બીજું ફંક્શન રન કરે (દા.ત. `sum(10)(20)(30)()`).

૪૨. Prototypal Inheritance & Chains

- **વિગતવાર સમજૂતી:** જાવાસ્ક્રિપ્ટનો વારસો નિયમ. આમાં એક ઓબ્જેક્ટ પાસે પ્રોપર્ટી ન હોય, તો તે પોતાની પાછળ છુપાયેલી લિંક સાંકળ (Prototype Chain) દ્વારા તેના પિતા સમાન ઓબ્જેક્ટની અંદર જઈને એ ફંક્શન કે ડેટા શોધીને વાપરી શકે છે.

૪૩. `__proto__` (Dunder Proto) vs prototype

- **વિગતવાર સમજૂતી:** `__proto__` એ દરેક સિંગલ ઓબ્જેક્ટની અંદર રહેલો અસલી જીવતો લિંક વાયર છે જે પેરેન્ટ સાથે કનેક્ટ રાખે છે. `prototype` એ ફક્ત Constructor ફંક્શન્સ કે ક્લાસ પાસે રહેલો એક ખાલી નકશો (Blueprint) છે જેમાં નવો લોજિક કોડ સ્ટોર થાય છે.

૪૪. JavaScript Polyfills & Shims

- **વિગતવાર સમજૂતી:** જો કોઈ કસ્ટમરના જૂના બ્રાઉઝરમાં આધુનિક જાવાસ્ક્રિપ્ટની કોઈ નવી મેથડ (જેમ કે `.includes()`) ઇન-બિલ્ટ હાજર ન હોય, તો બ્રાઉઝર કેશ ન થાય તે માટે આપણે જાતે ઝીરોથી એ ફંક્શન કોડ લખીને બ્રાઉઝરના પ્રોટોટાઇપમાં ઘુસાડી દઈએ છીએ, એ સપોર્ટ કોડને **Polyfill** કહેવાય છે.

૪૫. ESM Static Tree vs CommonJS (CJS) Dynamic Layouts

- **વિગતવાર સમજૂતી:** CommonJS (`require()`) જૂની પદ્ધતિ છે જે Synchronous અને Dynamic છે (મેઇન થ્રેડ બ્લોક કરે છે). ESM (`import/export`) આધુનિક સ્ટાન્ડર્ડ છે જે Asynchronous અને Static

છે, આના લીધે V8 એન્જિન કોડ રન કરતા પહેલા જ આખો નકશો સમજીને નકામો કોડ ઓટો-ડીલીટ (Tree Shaking) કરી શકે છે.

૪૬. "use strict"; (Strict Mode Engine)

- **વિગતવાર સમજૂતી:** ફાઇલની સૌથી ઉપર આ લાઇન લખતા જ V8 એન્જિન સુપર-સિક્યોર કડક મોડમાં આવી જાય છે. તે નાની ભૂલો, ડેટા લીક, અને Accidental Globals (કીવર્ડ વગર વેરીએબલ બનાવવા) ને તુરંત જ લાલ એરર આપીને ઓટો-બ્લોક કરી દે છે.



PART 5: BROWSER ENVIRONMENT & CLIENT STORAGE (બ્રાઉઝર સ્ટોરેજ)

૪૭. DOM Reflow / Layout & Paint Performance Overhead

- **વિગતવાર સમજૂતી:** સાદા જાવાસ્ક્રિપ્ટથી જ્યારે આપણે પેજ બદલીએ, ત્યારે બ્રાઉઝરે આખી વેબસાઇટના લુક, સાઇઝ અને કલરની ફરીથી ગણતરી કરવી પડે છે, જેને **Reflow** અને **Paint** કહેવાય છે. આ બ્રાઉઝર માટે બહુ જ મોંઘી (Slow) પ્રોસેસ છે, આના જ બચાવ માટે રીએક્ટ પોતાનો **Virtual DOM** વાપરે છે.

૪૮. Event Capturing vs Bubbling (ઇવેન્ટ વહેવાનો નિયમ)

- **વિગતવાર સમજૂતી:** જ્યારે તમે કોઈ ચાઇલ્ડ બટન ક્લિક કરો, ત્યારે ઇવેન્ટ પહેલા ઉપરથી નીચે ચાઇલ્ડ તરફ વહે છે જેને Capturing કહેવાય છે, અને બટન દબાયા પછી ઇવેન્ટ પાછી ચાઇલ્ડથી ઉપર પેરેન્ટ તરફ વહે છે જેને Bubbling કહેવાય છે.

૪૯. localStorage (લોકલ સ્ટોરેજ તિજોરી)

- **વિગતવાર સમજૂતી:** બ્રાઉઝરની અંદર આપેલી **5 MB** ની એવી કાયમી કી-વેલ્યુ સ્ટોરેજ જગ્યા, જેમાં સેવ થયેલો ડેટા બ્રાઉઝર બંધ કરવા કે કમ્પ્યુટર રીસ્ટાર્ટ કરવા છતાં ક્યારેય પણ ઓટોમેટિક ડીલીટ થતો નથી.

૫૦. sessionStorage (સેશન સ્ટોરેજ તિજોરી)

- **વિગતવાર સમજૂતી:** આ પણ **5 MB** ની જગ્યા આપે છે, પણ આની વેલિડિટી માત્ર એ સિંગલ બ્રાઉઝર ટેબ (Session) પૂરતી જ હોય છે. જેવું યુઝર એ ટેબ બંધ કરી દેશે, એટલે આખો ડેટા ઓટોમેટિક રેમમાંથી સાફ થઈ જશે.

૫૧. Document Cookies (HttpOnly HTTP Headers)

- **વિગતવાર સમજૂતી:** **4 KB** ની બહુ જ નાની સ્ટોરેજ સિસ્ટમ. આની ખાસિયત એ છે કે બ્રાઉઝર જ્યારે પણ સર્વર પર કોઈ નવી API રિક્વેસ્ટ મોકલે, ત્યારે કુકીઝનો ડેટા રિક્વેસ્ટ હેડરની સાથે આપોઆપ બેકગ્રાઉન્ડ સર્વર પર વહેતો જાય છે. HttpOnly લોક લગાવવાથી જાવાસ્ક્રિપ્ટનો કોઈ કોડ તેને ક્યારેય ચોરી શકતો નથી.

૫૨. IndexedDB (બ્રાઉઝર નોએસક્યુએલ ડેટાબેઝ)

- **વિગતવાર સમજૂતી:** બ્રાઉઝરની અંદર આપેલો એક આખો કમ્પીટ, અત્યંત પાવરફુલ **Transactional NoSQL Database**. તે 100% Asynchronous (Non-blocking) છે અને હાર્ડિસ્ટના આધારે **GB સુધીનો** વિશાળ ડેટા સ્ટોર કરવાની છૂટ આપે છે, જેનાથી આખી વેબસાઇટ ઇન્ટરનેટ વગર (Offline PWA Mode) ચાલી શકે છે.

૫૩. MutationObserver (લાઈવ DOM ચોકીદાર)

- **વિગતવાર સમજૂતી:** બ્રાઉઝરના અસલી DOM Tree ની અંદર થતા નાનામાં નાના ફેરફાર (જેમ કે કોઈ હેકરે રન-ટાઇમ પર નવો HTML ટેગ ઘુસાડ્યો કે ક્લાસ નેમ બદલ્યું), તેને લાઈવ ટ્રેક કરીને સેકન્ડમાં સેન્ટ્રલ બેકએન્ડ એલર્ટ ફાયર કરવાનું અદ્યતન ઇન-બિલ્ટ મોનિટરિંગ એન્જિન.

ચાલો, આપણે javascript.info ના તમામ સત્તાવાર પ્રકરણો અને આપણા **Advanced JS Master Radar Syllabus** ના એકેએક લો-લેવલ કન્સેપ્ટ્સને એકસાથે જોડીને, દુનિયાનો સૌથી મોટો અને સંપૂર્ણ **Mega Unique JavaScript Master Syllabus** તૈયાર કરીએ [image_5IPcLe.png].

અહીં દરેક સિંગલ ટર્મ અને ટોપિકને સેપરેટ (જુદી) લાઇન પર મૂકીને, તેની સામે એકદમ સરળ ગુજરાતી ભાષામાં વિગતવાર અને સખત આંતરિક સમજૂતી પ્રસ્તુત છે:

PART 1: THE JAVASCRIPT LANGUAGE FOUNDATIONS (ભાષાનો પાયો)

૧. JavaScript Engines (જાવાસ્ક્રિપ્ટ એન્જિન્સ)

ગૂગલ ક્રોમનું V8, ફાયરફોક્સનું SpiderMonkey અને સફારીનું JavaScriptCore એવા સોફ્ટવેર પ્રોગ્રામ્સ છે જે બ્રાઉઝરમાં જાવાસ્ક્રિપ્ટ કોડને કમ્પ્યુટરના પ્રોસેસર સમજી શકે તેવા મશીન કોડ (0 અને 1) માં ફેરવે છે.

૨. Just-In-Time (JIT) Compilation (જસ્ટ-ઇન-ટાઇમ કમ્પાઇલેશન)

આ એક આધુનિક ટેકનોલોજી છે જે કોડ રન થતો હોય તેની સાથે જ (On-the-fly) તેને ઓપ્ટિમાઇઝ્ડ મશીન કોડમાં કમ્પાઇલ કરે છે, જેનાથી જાવાસ્ક્રિપ્ટની સ્પીડ ૧૦૦ ગણી વધી જાય છે.

૩. Ignition Interpreter (ઇગ્નિશન ઇન્ટરપ્રિટર)

V8 એન્જિનનો આ પ્રથમ ગેટ છે. તે કોડ લોડ થતાં જ કમ્પાઇલેશનની રાહ જોયા વગર તેને તાત્કાલિક **Bytecode** માં ફેરવીને પ્રોગ્રામને સેકન્ડના હજારમા ભાગમાં ચાલુ કરી દે છે.

૪. TurboFan Optimizer (ટર્બોફેન ઓપ્ટિમાઇઝર)

V8 નો આ સુપર-કમ્પાઇલર ભાગ છે. પ્રોગ્રામ ચાલુ હોય ત્યારે જ ફંક્શન વારંવાર રન થતું હોય (Hot Code), તેને આ મશીન સીધું જ હાર્ડકોર મશીન કોડમાં કન્વર્ટ કરીને પ્રોસેસર પર રોકેટ સ્પીડે રન કરે છે.

૫. Abstract Syntax Tree (AST) (એબ્સ્ટ્રેક્ટ સિન્ટેક્સ ટ્રી)

V8 એન્જિન જ્યારે તમારો કોડ વાંચે છે, ત્યારે તે તેના લખાણને તોડીને મેમરીમાં એક વૃક્ષ જેવું બાઇનરી માળખું બનાવે છે, જેને AST કહેવાય છે, આના આધારે જ આગળ બાઇટકોડ બને છે.

૬. Code Deoptimization (ડી-ઓપ્ટિમાઇઝેશન બગ)

જો ટર્બોફેન કોઈ ફંક્શનને નંબર ડેટા ટાઇપ સમજીને ઓપ્ટિમાઇઝ કર્યું હોય, અને અચાનક યુઝર તેમાં સ્ટ્રિંગ પાસ કરી દે, તો એન્જિન કન્ફ્યુઝ થઈને ઓપ્ટિમાઇઝ કોડ ફેંકી દે છે અને પાછો સ્વો મોડ પર આવી જાય છે.

૭. Variables (વેરીએબલ્સ)

કમ્પ્યુટરની RAM મેમરીમાં ડેટા સાચવવા માટેનું એક નાનું કન્ટેનર (બોક્સ).

૮. let keyword (લેટ કીવર્ડ)

Block-Scoped (ફક્ત ક્લોઝ બ્રેકેટ { } ની અંદર જ ચાલશે). આમાં કિંમત બદલી (Re-assign) શકાય છે, પણ એ જ બ્લોકમાં ફરીથી નવો વેરીએબલ બનાવી (Re-declare) શકાતો નથી.

૯. const keyword (કોન્સ્ટ કીવર્ડ)

Constant (સ્થિર) કિંમતો માટે. આની કિંમત આખા પ્રોગ્રામમાં ક્યારેય પણ બદલાવી શકાતી નથી.

૧૦. var keyword (વેર કીવર્ડ)

Function-Scoped (ક્લોઝ બ્રેકેટ્સના નિયમો તોડીને બહાર લીક થઈ જાય છે). અજાણતા એક જ નામનો વેરીએબલ ગમે તેટલી વાર ફરીથી બનાવી શકાય છે જે મોટો ભગ લાવે છે.

૧૧. Primitive Types (પ્રિમિટિવ પ્રકારો)

એવો સાદો સિંગલ વેલ્યુ વાળો ડેટા જેને અંદરથી તોડી ન શકાય.

૧૨. undefined value

જ્યારે વેરીએબલ તો બનાવીએ પણ તેની અંદર કોઈ કિંમત ન મૂકીએ ત્યારે સિસ્ટમ તેને આપોઆપ 'undefined' સેટ કરે છે.

૧૩. null value

પ્રોગ્રામર દ્વારા જાણીજોઈને સેટ કરેલો શૂન્યાવકાશ (ખોટી વેલ્યુ). typeof null ચેક કરવા પર તે object આપે છે, જે જાવાસ્ક્રિપ્ટની પ્રથમ વર્જનથી ચાલી આવતી ટેકનિકલ ભૂલ છે.

૧૪. Symbol data type

ES6 નું ફીચર છે. મેમરીમાં ૧૦૦% એકદમ અનોખો (Unique) આઈડેન્ટિફાયર ટોકન બનાવવા આનો ઉપયોગ થાય છે.

૧૫. BigInt data type

સાદા Number ટાઇપની લિમિટ કરતાં બહુ જ મોટા આંકડાઓ (શેર બજાર કે સાયન્ટિફિક ગણતરીઓ) સાચવવા માટે આંકડાની પાછળ 'n' મૂકવો.

૧૬. Arithmetic Operators

ચિહ્નો: +, -, *, /, % (ભાગાકારની શેષ શોધવા), ** (ઘાત ગણવા માટે).

૧૭. Assignment Operators

ચિહ્નો: =, +=, -=, *= (ટૂંકી રીત). દા.ત. x += 5 એટલે કે x = x + 5.

૧૮. Logical && Operator (AND ગેટ)

શોર્ટ-સર્કિટ પર કામ કરે છે. જો આપેલી બધી જ શરતો સાચી (true) હશે તો જ આગળ વધશે.

૧૯. Logical || Operator (OR ગેટ)

આપેલી બધી શરતોમાંથી કોઈ પણ એક સિંગલ શરત પણ સાચી પડી જશે તો મંજૂરી આપી દેશે.

૨૦. Logical ! Operator (NOT ગેટ)

લોજિકને કાયની જેમ ઉલટાવી દે છે (સાચાનું ખોટું અને ખોટાનું સાચું કરવું).

૨૧. Comparison Operators

ચિહ્નો: >, <, >=, <=, !=, ==.

૨૨. == loose equality

ડેટા ટાઇપને ઇગ્નોર કરીને ફક્ત અંદરની કિંમત સરખાવે છે. (દા.ત. 5 == "5" ટ્રુ આપશે).

૨૩. === strict equality

પ્રોફેશનલ ઇન્ડસ્ટ્રીયલ સ્ટાન્ડર્ડ. આ કિંમત અને ડેટા ટાઇપ બંને ૧૦૦% સેમ હોવા પર જ સાચું (true) આપે છે.

૨૪. Implicit vs Explicit Coercion

Implicit એટલે જાવાસ્ક્રિપ્ટ એન્જિન આપોઆપ રન-ટાઇમ પર ટાઇપ બદલી નાખે (દા.ત. 5 + "5" = "55").

Explicit એટલે ડેવલપર જાતે ફંક્શન વાપરીને ટાઇપ બદલે (દા.ત. Number("100")).

૨૫. if-else statement

શરતના બુલિયન પરિણામ (true/false) ના આધારે કોડના રસ્તાઓ અલગ કરવા.

૨૬. switch-case statement

એક જ વેરીએબલની મલ્ટિપલ ફિક્સ કિંમતો સરખાવવા માટે. આમાં દરેક કેસ પૂરો થતાં 'break;' મૂકવો ફરજિયાત છે.

૨૭. Ternary Operator ? :

સાદા if-else નું સિંગલ લાઇન શોર્ટકટ રૂપ. રીએક્ટમાં કન્ડિશનલ રેન્ડરિંગ માટે આ સૌથી વધુ વપરાય છે. let msg = isLoggedIn ? "Welcome" : "Login";

૨૮. for loop

જ્યારે બબર હોય કે ચક્ર કેટલી વાર ફેરવવું છે. (Initialization; Condition; Increment).

૨૯. while loop

શૂરુઆતની કે ફિક્સ લિમિટ વગરનું શરતી ચક્ર. જ્યાં સુધી શરત true રહેશે ત્યાં સુધી અનંત ચાલ્યા કરશે (કાઉન્ટર બદલવો ફરજિયાત છે).

૩૦. do-while loop

Exit-Controlled. શરત સાચી હોય કે ખોટી, પ્રોગ્રામને ઓછામાં ઓછો ૧ વાર તો રન કરશે જ.

૩૧. for...of loop

એરે (Arrays) અને ઇટરેબલ્સની અંદરથી ઇન્ડેક્સ નંબરની માથાફૂટ વગર સીધી જ અસલી Values બહાર કાઢવા માટે.

૩૨. for...in loop

ઓબ્જેક્ટ્સ (Objects) ની અંદર રહેલી બધી જ Keys (ચાવીઓ) ને સ્કેન કરવા માટે. આનો ઉપયોગ એરે પર ક્યારેય ન કરવો.

૩૩. Function Declaration

હોઇસ્ટિંગ (High Hoisting) સપોર્ટ કરે છે. ફંક્શન બન્યા પહેલા પણ ઉપરની લાઇનમાં તેને કોલ કરી શકાય છે.

૩૪. Function Expression

ફંક્શનને વેરીએબલમાં લોક કરવું. આમાં હોઇસ્ટિંગ કામ કરતું નથી.

૩૫. Arrow Functions (=>)

સુપર શોર્ટકટ સિન્ટેક્સ. આની પાસે પોતાનો કોઈ 'this' કીવર્ડ હોતો નથી, તે રીએક્ટ ઇવેન્ટ્સ અને લેક્સિકલ બાઇન્ડિંગ માટે બેસ્ટ છે.

૩૬. Parameters vs Arguments

Parameters એટલે ફંક્શન ડિઝાઇન કરતી વખતે કૌંસમાં લખેલા કાલ્પનિક નામો (Variables). Arguments એટલે ફંક્શન રન કરતી વખતે અસલી મોકલેલી કિંમતો (Actual Values).

૩૭. Default Parameters

જો બહારથી આર્ગ્યુમેન્ટ ન આવે, તો ફંક્શન કેશ ન થાય તે માટે પેરામીટરને પહેલેથી એક સુરક્ષિત ડિફોલ્ટ વેલ્યુ આપી દેવી.

૩૮. Object Literals Basics

કર્લી બ્રેકેટ્સ { } નો ઉપયોગ કરીને કી-વેલ્યુ પેરના સ્વરૂપમાં ડેટા સાચવવાનો મીની-ડેટાબેઝ ઓબ્જેક્ટ.

૩૯. Dot vs Bracket Notation

Dot એટલે user.name (સાદી રીત). Bracket એટલે user["current city"] (સ્પેસ વાળી કી અથવા બીજા વેરીએબલથી ડાયનેમિકલી ડેટા રીડ કરવાની રીત).

૪૦. Array Base Methods

push() છેલ્લે ઉમેરે, pop() છેલ્લું ડીલીટ કરે, unshift() શરૂઆતમાં ડેટા ઘુસાડે (સ્વો મેથડ), shift() પહેલો ડેટા ઉડાવી દે.

PART 2: OBJECTS & MEMORY MANAGEMENT (ઓબ્જેક્ટ્સ અને મેમરી)

૪૧. Call Stack Memory (કોલ સ્ટેક મેમરી)

V8 એન્જિનની અંદર રહેલું એક સુપર-ફાસ્ટ, ફિક્સ સાઇઝનું માળખું જે LIFO (Last In, First Out) નિયમ પર કામ કરે છે. તે પ્રોગ્રામના ફંક્શન ઓર્ડર અને સાદા પ્રિમિટિવ વેરીએબલ્સને સાચવે છે.

૪૨. Heap Allocation Memory (હીપ મેમરી)

રેમ (RAM) ની અંદર આવેલી એક બહુ જ મોટી, અનસ્ટ્રક્ચર્ડ જગ્યા જ્યાં બધા જ જટિલ અને મોટા ડેટા ઓબ્જેક્ટ્સ (Objects, Arrays) નો અસલી ભારે ભારે ડેટા સ્ટોર થાય છે.

૪૩. Memory Address Reference (મેમરી પોઇન્ટર)

ઓબ્જેક્ટનો અસલી ડેટા Heap માં સેવ થાય છે, પણ એ જગ્યાનું જે સરનામું (Address Pointer) હોય છે, તે સાઇઝમાં નાનું હોવાથી આપણો Call Stack તેને પોતાની ફ્રેમમાં કોપી તરીકે સાચવી રાખે છે.

૪૪. Value vs Reference Types

Primitives (Number, String) એ Value Types છે જેની નવી સ્વતંત્ર કોપી બને છે. Objects અને Arrays એ Reference Types છે જેમાં અસલી ડેટા કોપી નથી થતો, ફક્ત મેમરી સરનામું શેર થાય છે.

૪૫. Shallow Copy (Spread Operator ...)

આ ઓબ્જેક્ટના પહેલા લેયરને તો નવી મેમરીમાં સેફ કોપી કરી લે છે, પણ જો ઓબ્જેક્ટની અંદર પણ બીજો કોઈ નેસ્ટેડ (Nested) ઓબ્જેક્ટ હોય, તો તેનું અંદરનું મેમરી સરનામું જૂનું જ રહે છે.

૪૬. Deep Copy via structuredClone()

ઓરિજિનલ ઓબ્જેક્ટ સાથેથી મેમરીના તમામ વાયરો કાયમ માટે તોડીને, ગમે તેટલો ઊંડો નેસ્ટેડ ડેટા હોય તો પણ તેની ૧૦૦% બ્રાન્ડ-ન્યુ કોપી બનાવવાની સત્તાવાર આધુનિક મેથડ.

૪૭. V8 Garbage Collection (ગાર્બેજ કલેક્શન)

બેકગ્રાઉન્ડમાં ચાલતો એક ઓટોમેટિક પ્રોગ્રામ જે મેમરી (Heap) માં એવા ઓબ્જેક્ટ્સને શોધીને રેમમાંથી કાયમ માટે ડીલીટ કરી નાખે છે જેનો પ્રોગ્રામમાં કોઈ પણ જગ્યાએ વાયર કે રેફરન્સ બાકી રહ્યો નથી.

૪૮. Scavenger GC Phase (Young Generation)

ન્યૂ સ્પેસ (New Space) ને સાફ કરવાનો હાઇ-સ્પીડ અલ્ગોરિધમ. તે From-Space માંથી જીવતા ઓબ્જેક્ટ્સ ફાસ્ટ સ્કેન કરીને બાજુના ખાલી To-Space માં કોપી કરી લે છે અને બાકીનો કચરો એક ઝાટકે સાફ કરે છે.

૪૯. Mark-Sweep GC Phase (Old Generation)

ઓલ્ડ સ્પેસ (Old Space) ને સાફ કરવાનો મોટો પ્રોગ્રામ. તે ગ્લોબલ રૂટ્સથી મુસાફરી શરૂ કરીને જીવતા ઓબ્જેક્ટ પર Mark (નિશાન) લગાવે છે અને બાકીના અન-રેફરન્સ કચરાને Sweep (Delete) કરી મેમરી મુક્ત કરે છે.

૫૦. Memory Leak Vectors (મેમરી લીક)

જ્યારે કોઈ ઓબ્જેક્ટનું કામ પૂરું થઈ જાય, છતાં અજાણતા તેનો કોઈ નાનો વાયર ગ્લોબલ સ્કોપ કે ટાઇમર સાથે જોડાયેલો રહી જાય, ત્યારે ગાર્બેજ કલેક્ટર તેને કાયમ માટે ડીલીટ નથી કરી શકતો, જેનાથી રેમ ભરાતી જાય છે.

૫૧. Object.freeze() (ઓબ્જેક્ટ ફ્રીઝ)

ઓબ્જેક્ટની બધી કી ના ઇન્ટરનલ ઓબ્જેક્ટ ડિસ્ક્રિપ્ટર્સ writable: false અને configurable: false કરી દે છે, જેથી પ્રોગ્રામમાં કોઈ નવી કી ઉમેરી, જૂની ડીલીટ કે અંદરની વેલ્યુ બદલી શકાતી નથી.

૫૨. Object.seal() (ઓબ્જેક્ટ સીલ)

આ ઓબ્જેક્ટની બધી પ્રોપર્ટીઝના configurable: false કરી દે છે પણ writable ને true રાખે છે. આનાથી ઓબ્જેક્ટમાં નવી કી પ્લસ કે માઇનસ નથી કરી શકાતી, પણ જે જૂની કી હાજર છે તેની અંદરનો વેલ્યુ ડેટા બદલી શકાય છે.

૫૩. Object Property Descriptors (પ્રોપર્ટી ડિસ્ક્રિપ્ટર્સ)

ઓબ્જેક્ટની દરેક કી ની પાછળ છુપાયેલા ૩ સિક્યોરિટી વેલ્યુ લોક—writable (કિંમત બદલવી કે નહીં), enumerable (લૂપમાં દેખાડવી કે હાઇડ રાખવી), અને configurable (ડીલીટ કરવા દેવી કે લોક રાખવી).

૫૪. JavaScript Proxies & Reflect API (પ્રોક્સી પ્રોટેક્શન)

ઓરિજિનલ ઓબ્જેક્ટની આજુબાજુ બનેલું એક પાવરફુલ સુરક્ષા કવચ. બહારનો કોઈ કોડ ઓબ્જેક્ટને સીધો અડી શકતો નથી, વચ્ચે રહેલા Get/Set Traps તેને હવામાં પકડીને સિક્યોરિટી ચેક કરીને જ ડેટા બદલવા કે જોવા દે છે.

🌀 PART 3: ADVANCED RUNTIME CONCURRENCY (ઇવેન્ટ લૂપ અને અસિંક)

૫૫. The Event Loop Mechanics (ઇવેન્ટ લૂપનું ચક્ર)

libuv C++ લાઇબ્રેરી દ્વારા ચાલતું એક એવું મેકેનિઝમ જે સિંગલ-થ્રેડેડ જાવાસ્ક્રિપ્ટને Non-blocking પાવર આપે છે. તે Call Stack ખાલી થતાં જ જાગે છે અને અસિંક્રોનસ કામોના કોલબેક્સને લાઇનસર પ્રોસેસ કરે છે.

૫૬. The 6 Loop Phases (ઇવેન્ટ લૂપના ૬ ચક્રો)

લૂપનો દરેક રાઉન્ડ (Tick) આ ૬ ફેઝમાંથી પસાર થાય છે: Timers (setTimeout) -> Pending OS Callbacks -> Idle/Prepare (સિસ્ટમ ઇન્ટરનલ) -> Poll (નવો નેટવર્ક ડેટા બ્લોકિંગ) -> Check (setImmediate) -> Close Callbacks (કનેક્શન ક્લીનઅપ).

૫૭. Microtask Priority Hijacking (માઇક્રોટાસ્ક વીઆઇપી ક્યુ)

આ ઇવેન્ટ લૂપની સૌથી શક્તિશાળી VIP લાઇન છે જેમાં બધા Promises (.then/.catch) અને async/await ની કોડ ઊભા રહે છે. લૂપ ગમે તે ફેઝમાં હોય, પણ જ્યાં સુધી આ માઇક્રોટાસ્ક ક્યુ પૂરેપૂરી ખાલી ન થાય, ત્યાં સુધી તે આગળ વધી શકતું નથી.

૫૮. process.nextTick() Scheduling (સુપર કમાન્ડર)

Node.js નો આ સર્વોચ્ચ કમાન્ડર છે જેનો પાવર પ્રોમિસ કરતાં પણ પહેલો છે. ઇવેન્ટ લૂપ પોતાનો ચાલુ ફેઝ પૂરો કરીને આગલા સ્ટેપ પર પગ મૂકે તેની બરાબર વચ્ચે આ nextTick નો કોડ સૌથી પહેલા રન કરવો જ પડે છે.

૫૯. Stack Overflow Vectors (મેક્સિમમ કોલ સ્ટેક કેશ)

જ્યારે કોઈ ફંક્શન પોતાને ને પોતાને જ કોઈ પણ અટકવાના નિયમ વગર વારંવાર કોલ કર્યા કરે (Infinite Recursion), ત્યારે Call Stack પૂરેપૂરો ઉપર સુધી ભરાઈ જાય છે અને એન્જિન Maximum call stack size exceeded ની એરર આપે છે.

૬૦. Async / Await Generator Pause Engine

જ્યારે કોડમાં await આવે, ત્યારે V8 એન્જિન આખું સર્વર જામ નથી કરતું. તે બેકગ્રાઉન્ડમાં ફંક્શનના એક્ઝિક્યુશનને ત્યાં જ Pause (અટકાવી) દે છે. જવાબ આવતા જ માઇક્રોટાસ્ક ક્યુમાંથી કોડ પાછો ત્યાંથી જ Resume થાય છે.

⚙️ PART 4: COMPILATION, CLOSURES & OOP (આર્કિટેક્ચર નિયમો)

૬૧. [[Scopes]] Closure Memory Object

જ્યારે પેરેન્ટ ફંક્શન રન થઈને કોલ સ્ટેકમાંથી નાશ પામે, છતાં ચાઇલ્ડ ફંક્શન તેના વેરીએબલ્સ વાપરતું હોય, ત્યારે V8 એન્જિન એ વેરીએબલ્સને સ્ટેકમાંથી ઉઠાવીને Heap મેમરીમાં એક ખાસ પ્રાઇવેટ ઓબ્જેક્ટમાં લોક કરી દે છે, જેને [[Scopes]] Closure કહેવાય છે.

૬૨. Data Encapsulation (ડેટા કેપ્સ્યુલેશન)

ઇમ્પોર્ટન્ટ વેરીએબલ્સને ફંક્શનની અંદર છુપાવીને (Private કરીને) બહારની દુનિયાના કોડથી પ્રોટેક્ટ કરવા. ડેટા બદલવા કે જોવા માટે આપણે માત્ર કસ્ટમ સુરક્ષિત ગેટ્સ (Getters and Setters) જ ઓપન રાખીએ છીએ.

૬૩. Factory Functions Design Pattern

સોફ્ટવેર આર્કિટેક્ચરની એવી સુંદર પેટર્ન જેમાં કોઈ class કે new કીવર્ડ વગર, સાદા ફંક્શનની અંદર ક્લોઝર સેટ કરીને ઓટોમેટિક એક સિક્યોર કસ્ટમ ઓબ્જેક્ટ મેન્યુફેક્ચર (બનાવીને) બહાર મોકલવામાં આવે છે.

૬૪. Function Currying & Infinite Chains

એકસાથે બધી આર્ગ્યુમેન્ટ્સ લેવાના બદલે ફંક્શનને એવા ટુકડાઓમાં ફેરવવું જ્યાં દરેક ફંક્શન માત્ર ૧ જ આર્ગ્યુમેન્ટ લે અને અંદરથી બીજું ફંક્શન રન કરે (દા.ત. `sum(10)(20)(30)()`).

૬૫. Prototypal Inheritance & Chains

જાવાસ્ક્રિપ્ટનો વારસો નિયમ. આમાં એક ઓબ્જેક્ટ પાસે પ્રોપર્ટી ન હોય, તો તે પોતાની પાછળ છુપાયેલી લિંક સાંકળ (Prototype Chain) દ્વારા તેના પિતા સમાન ઓબ્જેક્ટની અંદર જઈને એ ફંક્શન કે ડેટા શોધીને વાપરી શકે છે.

૬૬. __proto__ (Dunder Proto) vs prototype

`__proto__` એ દરેક સિંગલ ઓબ્જેક્ટની અંદર રહેલો અસલી જીવતો લિંક વાયર છે જે પેરેન્ટ સાથે કનેક્ટ રાખે છે. `prototype` એ ફક્ત Constructor ફંક્શન્સ કે ક્લાસ પાસે રહેલો એક ખાલી નકશો (Blueprint) છે જેમાં નવો લોજિક કોડ સ્ટોર થાય છે.

૬૭. JavaScript Polyfills & Shims

જો કોઈ કસ્ટમરના જૂના બ્રાઉઝરમાં આધુનિક જાવાસ્ક્રિપ્ટની કોઈ નવી મેથડ (જેમ કે `.includes()`) ઇન-બિલ્ટ હાજર ન હોય, તો બ્રાઉઝર કેશ ન થાય તે માટે આપણે જાતે ઝીરોથી એ ફંક્શન કોડ લખીને બ્રાઉઝરના પ્રોટોટાઇપમાં ઘુસાડી દઈએ છીએ, એ સપોર્ટ કોડને Polyfill કહેવાય છે.

૬૮. ESM Static Tree vs CommonJS (CJS)

CommonJS (`require()`) જૂની પદ્ધતિ છે જે Synchronous અને Dynamic છે. ESM (`import/export`) આધુનિક સ્ટાન્ડર્ડ છે જે Asynchronous અને Static છે, આના લીધે V8 એન્જિન કોડ રન કરતા પહેલા જ આખો નકશો સમજીને નકામો કોડ ઓટો-ડીલીટ (Tree Shaking) કરી શકે છે.

૬૯. "use strict"; (Strict Mode Engine)

ફાઇલની સૌથી ઉપર આ લાઇન લખતા જ V8 એન્જિન સુપર-સિક્યોર કડક મોડમાં આવી જાય છે. તે નાની ભૂલો, ડેટા લીક, અને Accidental Globals ને તુરંત જ લાલ એરર આપીને ઓટો-બ્લોક કરી દે છે.



PART 5: BROWSER ENVIRONMENT & CLIENT STORAGE (બ્રાઉઝર સ્ટોરેજ)

૭૦. DOM Reflow / Layout & Paint Overhead

સાદા જાવાસ્ક્રિપ્ટથી જ્યારે આપણે પેજ બદલીએ, ત્યારે બ્રાઉઝરે આખી Parental વેબસાઇટના લુક, સાઇઝ અને કલરની ફરીથી ગણતરી કરવી પડે છે, જેને Reflow અને Paint કહેવાય છે. આ બ્રાઉઝર માટે બહુ જ સ્વો પ્રોસેસ છે, આના જ બચાવ માટે રીએક્ટ પોતાનો Virtual DOM વાપરે છે.

૭૧. Event Capturing vs Bubbling

જ્યારે તમે કોઈ યાદશ્લ બટન ક્લિક કરો, ત્યારે ઇવેન્ટ પહેલા ઉપરથી નીચે યાદશ્લ તરફ વહે છે જેને Capturing કહેવાય છે, અને બટન દબાયા પછી ઇવેન્ટ પાછી યાદશ્લથી ઉપર પેરેન્ટ તરફ વહે છે જેને Bubbling કહેવાય છે.

૭૨. localStorage (લોકલ સ્ટોરેજ તિજોરી)

બ્રાઉઝરની અંદર આપેલી 5 MB ની એવી કાયમી કી-વેલ્યુ સ્ટોરેજ જગ્યા, જેમાં સેવ થયેલો ડેટા બ્રાઉઝર બંધ કરવા કે કમ્પ્યુટર રીસ્ટાર્ટ કરવા છતાં ક્યારેય પણ ઓટોમેટિક ડીલીટ થતો નથી.

૭૩. sessionStorage (સેશન સ્ટોરેજ તિજોરી)

આ પણ 5 MB ની જગ્યા આપે છે, પણ આની વેલિડિટી માત્ર એ સિંગલ બ્રાઉઝર ટેબ (Session) પૂરતી જ હોય છે. જેવું યુઝર એ ટેબ બંધ કરી દેશે, એટલે આખો ડેટા ઓટોમેટિક રેમમાંથી સાફ થઈ જશે.

૭૪. Document Cookies (HttpOnly Headers)

4 KB ની બહુ જ નાની સ્ટોરેજ સિસ્ટમ. આની ખાસિયત એ છે કે બ્રાઉઝર જ્યારે પણ સર્વર પર કોઈ નવી API રિક્વેસ્ટ મોકલે, ત્યારે કુકીઝનો ડેટા આપોઆપ બેકએન્ડ સર્વર પર વહેતો જાય છે. HttpOnly લોક લગાવવાથી જાવાસ્ક્રિપ્ટનો કોઈ કોડ તેને ક્યારેય ચોરી શકતો નથી.

૭૫. IndexedDB (બ્રાઉઝર નોએસક્યુએલ ડેટાબેઝ)

બ્રાઉઝરની અંદર આપેલો એક આખો કમ્પીટ, અત્યંત પાવરફુલ Transactional NoSQL Database. તે 100% Asynchronous (Non-blocking) છે અને હાર્ડિસ્ટ્રક્ચર આધારે GB સુધીનો વિશાળ ડેટા સ્ટોર કરવાની છૂટ આપે છે, જેનાથી આખી વેબસાઇટ ઇન્ટરનેટ વગર (Offline Mode) ચાલી શકે છે.

૭૬. MutationObserver (લાઈવ DOM ચોકીદાર)

બ્રાઉઝરના અસલી DOM Tree ની અંદર થતા નાનામાં નાના ફેરફાર (જેમ કે કોઈ હેકરે રન-ટાઇમ પર નવો HTML ટેગ ઘુસાડ્યો કે ક્લાસ નેમ બદલ્યું), તેને લાઈવ ટ્રેક કરીને સેકન્ડમાં સેન્ટ્રલ બેકએન્ડ એલર્ટ ફાયર કરવાનું અદ્યતન ઇન-બિલ્ટ મોનિટરિંગ એન્જિન.

PART 1: CORE RUNTIME & ENGINE UNDERPINNINGS (એન્જિન આર્કિટેક્ચર)

૧. JavaScript Engines (જાવાસ્ક્રિપ્ટ એન્જિન્સ)

- **વિગતવાર સમજૂતી:** ગૂગલ ક્રોમનું V8, ફાયરફોક્સનું SpiderMonkey અને સફારીનું JavaScriptCore એવા શક્તિશાળી C++ પ્રોગ્રામ્સ છે જે આપણે લખેલા ટેક્સ્ટ કોડને કમ્પ્યુટરનું પ્રોસેસર સમજી શકે તેવા મશીન કોડ (0 અને 1) માં ફેરવે છે.

૨. Just-In-Time (JIT) Compilation (જસ્ટ-ઇન-ટાઇમ કમ્પાઇલેશન)

- **વિગતવાર સમજૂતી:** આ એક આધુનિક હાઇબ્રિડ ટેકનોલોજી છે જે ઇન્ટરપ્રિટર (લાઇન-બાય-લાઇન) અને કમ્પાઇલર (એકસાથે) બંનેની શક્તિઓ મિક્સ કરીને, કોડ રન થતો હોય તેની સાથે જ ઓન-ધ-ફ્લાય તેને રોકેટો સ્પીડ મશીન કોડમાં કન્વર્ટ કરે છે.

૩. Ignition Interpreter (ઇગ્નિશન ઇન્ટરપ્રિટર)

- **વિગતવાર સમજૂતી:** V8 એન્જિનનો આ પ્રથમ સિકેટ ગેટ છે. તે કોડ લોડ થતાં જ કમ્પાઇલેશનના ભારે ટાઇમનો વેસ્ટ કર્યા વગર તેને તાત્કાલિક Bytecode માં ફેરવીને એપ્લિકેશનને પલક ઝપકાવતા જ ચાલુ કરી દે છે.

૪. TurboFan Optimizer (ટર્બોફેન ઓપ્ટિમાઇઝર)

- **વિગતવાર સમજૂતી:** V8 નો આ અત્યંત બુદ્ધિશાળી કમ્પાઇલર ભાગ છે. પ્રોગ્રામ ચાલુ હોય ત્યારે જે ફંક્શન વારંવાર અને સતત રન થતું હોય (Hot Code), તેને આ મશીન સીધું જ હાર્ડકોર મશીન કોડમાં કન્વર્ટ કરીને પ્રોસેસર પર ડાયરેક્ટ રન કરે છે.

૫. Abstract Syntax Tree (AST) (એબ્સ્ટ્રેક્ટ સિન્ટેક્સ ટ્રી)

- **વિગતવાર સમજૂતી:** V8 એન્જિન જ્યારે તમારો કોડ વાંચે છે, ત્યારે તે તેના લખાણને તોડીને મેમરીમાં એક વૃક્ષ જેવું બાઇનરી ડેટા માળખું બનાવે છે, જેને AST કહેવાય છે. આના આધારે જ આગળ બાઇટકોડ બને છે.

૬. Code Deoptimization (ડી-ઓપ્ટિમાઇઝેશન)

- **વિગતવાર સમજૂતી:** જો ટર્બોફેન કોઈ ફંક્શનને ઇન્ટરનલી ખૂબ સ્પીડમાં રન કરવા માટે લોક કર્યું હોય, અને અચાનક યુઝર તેમાં ખોટો ડેટા ટાઇપ પાસ કરી દે, તો એન્જિન કન્ફ્યુઝ થઈને ઓપ્ટિમાઇઝ કોડ કચરાટોપલીમાં ફેંકી દે છે અને પાછો સ્વો મોડ પર આવી જાય છે.

૭. Execution Context (એક્ઝિક્યુશન કન્ટેક્સ્ટ)

- **વિગતવાર સમજૂતી:** જાવાસ્ક્રિપ્ટની મેમરીમાં બનતું એક એવું વર્ચ્યુઅલ રૂમ (બોક્સ) છે જેની અંદર અત્યારે કોડના કયા વેરીએબલ્સ લોડ છે, કઈ લાઇન રન થઈ રહી છે અને કયો સ્કોપ એક્ટિવ છે તેનો આખો લાઇવ નકશો સચવાય છે.

૮. Global Scope (ગ્લોબલ સીમાડો)

- **વિગતવાર સમજૂતી:** કોઈપણ ફંક્શન, ક્લાસ કે ક્વી બ્રેકેટ્સ { } ની બહાર બનેલો વેરીએબલ ગ્લોબલ સ્કોપમાં હોય છે, જે આખા પ્રોજેક્ટની ફાઇલમાં ગમે ત્યાંથી એક્સેસ અને મોડિફાઇ કરી શકાય છે.

૯. Script Scope (સ્ક્રીપ્ટ સ્કોપ)

- **વિગતવાર સમજૂતી:** ગ્લોબલ એરિયામાં બનેલા let અને const વેરીએબલ્સ આ ક્વચમાં સેવ થાય છે, જેથી તે આખા પેજ પરથી એક્સેસ તો થાય છે પણ ગ્લોબલ window ઓબ્જેક્ટની અંદર કોપી નથી થતા, જે હેકિંગ પ્રોટેક્શન આપે છે.

૧૦. Local / Function Scope (લોકલ સ્કોપ)

- **વિગતવાર સમજૂતી:** ફંક્શનની અંદર બનેલા વેરીએબલ્સ ફંક્શનની પોતાની પ્રાઇવેટ જેલમાં લોક હોય છે. ફંક્શનની બહાર આખા પ્રોગ્રામમાં તેને કોઈ જોઈ કે અડી શકતું નથી.

૧૧. Block Scope (બ્લોક સ્કોપ)

- **વિગતવાર સમજૂતી:** સાદા ક્વી બ્રેકેટ્સ { } (જેમ કે if કે for લૂપ) નો સ્કોપ. માત્ર let અને const જ આ નિયમ પાળે છે અને બ્લોક પૂરો થતાં જ રેમમાંથી ઓટોમેટિક નાશ પામે છે.

૧૨. Variable & Function Hoisting (હોઇસ્ટિંગ મિકેનિઝમ)

- **વિગતવાર સમજૂતી:** કોડ રન થાય તે પહેલા V8 એન્જિન બધા વેરીએબલ્સ અને ફંક્શન ડિક્લેરેશન્સને મેમરીમાં સૌથી ઉપર (Top) મોકલી દે છે. var હોઇસ્ટ થતાં undefined બને છે, જ્યારે ફંક્શન બન્યા પહેલા પણ રન થઈ શકે છે.

૧૩. Temporal Dead Zone (TDZ) (ટેમ્પોરલ ડેડ ઝોન)

- **વિગતવાર સમજૂતી:** let અને const હોઇસ્ટ થયા પછી જ્યાં સુધી કોડમાં પોતાની અસલી લાઇન પર ન પહોંચે, ત્યાં સુધી તે મેમરીમાં એક ગુપ્ત પ્રતિબંધિત એરિયામાં લોક રહે છે. આ ઝોનની અંદર તેને એક્સેસ કરતી જ ReferenceError આવે છે.

૧૪. Type Coercion (Implicit vs Explicit)

- **વિગતવાર સમજૂતી:** Implicit એટલે જાવાસ્ક્રિપ્ટ એન્જિન પોતાની મેળે ઓટોમેટિક ડેટા ટાઇપ બદલે (જેમ કે 5 + "5" = "55"). Explicit એટલે ડેવલપર પોતે જાણીજોઈને ઇન-બિલ્ટ ફંક્શન વાપરીને ટાઇપ બદલે (જેમ કે Number("10")).

૧૫. Loose Equality (==) vs Strict Equality (===)

- **વિગતવાર સમજૂતી:** Double Equals (==) ડેટા ટાઇપ જોયા વગર ફક્ત અંદરની કિંમત સરખાવે છે. જ્યારે Triple Equals (===) કિંમત અને ડેટા ટાઇપ બંને ૧૦૦% સેમ હોવા પર જ સાચું (true) આપે છે, જે ઇન્ડસ્ટ્રી સ્ટાન્ડર્ડ છે.

૧૬. Short-Circuit Evaluation (લોજિકલ શોર્ટ-સર્કિટ)

- **વિગતવાર સમજૂતી:** && ઓપરેટરને પહેલી જ શરત જો false મળી જાય, અથવા || ઓપરેટરને પહેલી શરત true મળી જાય, તો તે સમય બગાડ્યા વગર આગળનો કોડ ચેક કર્યા વગર ત્યાંથી જ આખું રિઝલ્ટ આપીને બહાર નીકળી જાય છે.

૧૭. Strict Mode ("use strict";)

- **વિગતવાર સમજૂતી:** ફાઇલની સૌથી ઉપર આ લાઇન લખતા જ V8 એન્જિન સુપર-સિક્યોર કડક મોડમાં આવી જાય છે. તે નાની ભૂલો, ડેટા લીક, અને Accidental Globals ને તુરંત જ લાલ અરર આપીને ઓટો-બ્લોક કરી દે છે.

PART 2: MEMORY MANAGEMENT & MEMORY TOPOLOGY (મેમરી આર્કિટેક્ચર)

૧૮. Call Stack Memory (કોલ સ્ટેક મેમરી)

- **વિગતવાર સમજૂતી:** V8 એન્જિનની અંદર રહેલું એક સુપર-ફાસ્ટ, ફિક્સ સાઇઝનું માળખું જે LIFO (Last In, First Out) નિયમ પર કામ કરે છે. તે પ્રોગ્રામના ફંક્શન ઓર્ડર અને સાદા પ્રિમિટિવ વેરીએબલ્સને સાચવે છે.

૧૯. Heap Allocation Memory (હીપ મેમરી)

- **વિગતવાર સમજૂતી:** રેમ (RAM) ની અંદર આવેલી એક બહુ જ મોટી, અનસ્ટ્રક્ચર્ડ જગ્યા જ્યાં બધા જ જટિલ અને મોટા ડેટા ઓબ્જેક્ટ્સ (Objects, Arrays) નો અસલી ભારે ભારે ડેટા સ્ટોર થાય છે.

૨૦. Memory Address Reference (મેમરી પોઇન્ટર)

- **વિગતવાર સમજૂતી:** ઓબ્જેક્ટનો અસલી ડેટા Heap માં સેવ થાય છે, પણ એ જગ્યાનું જે સરનામું (Address Pointer) હોય છે, તે સાઇઝમાં નાનું હોવાથી આપણો Call Stack તેને પોતાની ફ્રેમમાં કોપી તરીકે સાચવી રાખે છે.

૨૧. Value vs Reference Types

- **વિગતવાર સમજૂતી:** Primitives (Number, String) એ Value Types છે જેની નવી સ્વતંત્ર કોપી બને છે. Objects અને Arrays એ Reference Types છે જેમાં અસલી ડેટા કોપી નથી થતો, ફક્ત મેમરી સરનામું શેર થાય છે.

૨૨. Shallow Copy (Spread Operator ...)

- **વિગતવાર સમજૂતી:** આ ઓબ્જેક્ટના પહેલા લેયરને તો નવી મેમરીમાં સેફ કોપી કરી લે છે, પણ જો ઓબ્જેક્ટની અંદર પણ બીજો કોઈ નેસ્ટેડ (Nested) ઓબ્જેક્ટ હોય, તો તેનું અંદરનું મેમરી સરનામું જૂનું જ રહે છે.

૨૩. Deep Copy via structuredClone()

- **વિગતવાર સમજૂતી:** ઓરિજિનલ ઓબ્જેક્ટ સાથેથી મેમરીના તમામ વાયરો કાયમ માટે તોડીને, ગમે તેટલો ઊંડો નેસ્ટેડ ડેટા હોય તો પણ તેની ૧૦૦% બ્રાન્ડ-ન્યુ કોપી બનાવવાની સત્તાવાર આધુનિક મેથડ.

૨૪. V8 Garbage Collection (ગાર્બેજ કલેક્શન)

- **વિગતવાર સમજૂતી:** બેકગ્રાઉન્ડમાં ચાલતો એક ઓટોમેટિક પ્રોગ્રામ જે મેમરી (Heap) માં એવા ઓબ્જેક્ટ્સને શોધીને રેમમાંથી કાયમ માટે ડીલીટ કરી નાખે છે જેનો પ્રોગ્રામમાં કોઈ પણ જગ્યાએ વાયર કે રેફરન્સ બાકી રહ્યો નથી.

૨૫. Scavenger GC Phase (Young Generation)

- **વિગતવાર સમજૂતી:** ન્યૂ સ્પેસ (New Space) ને સાફ કરવાનો હાઇ-સ્પીડ અલ્ગોરિથમ. તે From-Space માંથી જીવતા ઓબ્જેક્ટ્સ ફાસ્ટ સ્કેન કરીને બાજુના ખાલી To-Space માં કોપી કરી લે છે અને બાકીનો કચરો એક ઝાટકે સાફ કરે છે.

૨૬. Mark-Sweep GC Phase (Old Generation)

- **વિગતવાર સમજૂતી:** ઓલ્ડ સ્પેસ (Old Space) ને સાફ કરવાનો મોટો પ્રોગ્રામ. તે ગ્લોબલ રુટ્સથી મુસાફરી શરૂ કરીને જીવતા ઓબ્જેક્ટ પર Mark (નિશાન) લગાવે છે અને બાકીના અન-રેફરન્સ કચરાને Sweep (Delete) કરી મેમરી મુક્ત કરે છે.

૨૭. Memory Leak Vectors (મેમરી લીક)

- **વિગતવાર સમજૂતી:** જ્યારે કોઈ ઓબ્જેક્ટનું કામ પૂરું થઈ જાય, છતાં અજાણતા તેનો કોઈ નાનો વાયર ગ્લોબલ સ્કોપ કે ટાઇમર સાથે જોડાયેલો રહી જાય, ત્યારે ગાર્બેજ કલેક્ટર તેને કાયમ માટે ડીલીટ નથી કરી શકતો, જેનાથી રેમ ભરાતી જાય છે.

૨૮. Object.freeze() (ઓબ્જેક્ટ ફ્રીઝ)

- **વિગતવાર સમજૂતી:** ઓબ્જેક્ટની બધી કી ના ઇન્ટરનલ ઓબ્જેક્ટ ડિસ્ક્રિપ્ટર્સ writable: false અને configurable: false કરી દે છે, જેથી પ્રોગ્રામમાં કોઈ નવી કી ઉમેરી, જૂની ડીલીટ કે અંદરની વેલ્યુ બદલી શકાતી નથી.

૨૯. Object.seal() (ઓબ્જેક્ટ સીલ)

- **વિગતવાર સમજૂતી:** આ ઓબ્જેક્ટની બધી પ્રોપર્ટીઝના configurable: false કરી દે છે પણ writable ને true રાખે છે. આનાથી ઓબ્જેક્ટમાં નવી કી પ્લસ કે માઇનસ નથી કરી શકાતી, પણ જે જૂની કી હાજર છે તેની અંદરનો વેલ્યુ ડેટા બદલી શકાય છે.

૩૦. Object Property Descriptors (પ્રોપર્ટી ડિસ્ક્રિપ્ટર્સ)

- **વિગતવાર સમજૂતી:** ઓબ્જેક્ટની દરેક કી ની પાછળ છુપાયેલા ૩ સિક્યોરિટી વેલ્યુ લોક—writable (ફિમત બદલવી કે નહીં), enumerable (લૂપમાં દેખાડવી કે હાઇડ રાખવી), અને configurable (ડીલીટ કરવા દેવી કે લોક રાખવી).

૩૧. JavaScript Proxies & Reflect API (પ્રોક્સી પ્રોટેક્શન)

- **વિગતવાર સમજૂતી:** ઓરિજિનલ ઓબ્જેક્ટની આજુબાજુ બનેલું એક પાવરફુલ સુરક્ષા કવચ. બહારનો કોઈ કોડ ઓબ્જેક્ટને સીધો અડી શકતો નથી, વચ્ચે રહેલા Get/Set Traps તેને હવામાં પકડીને સિક્યોરિટી ચેક કરીને જ ડેટા બદલવા કે જોવા દે છે.

PART 5: ADVANCED COMPILATION, CLOSURES & SCOPE (ચક્ર અને પ્રાયોરિટી)

૬૧. [[Scopes]] Closure Memory Object (ક્લોઝર મેમરી ઓબ્જેક્ટ)

જ્યારે પેરેન્ટ ફંક્શન રન થઈને કોલ સ્ટેકમાંથી નાશ પામે, છતાં ચાઇલ્ડ ફંક્શન તેના વેરીએબલ્સ વાપરતું હોય, ત્યારે V8 એન્જિન એ વેરીએબલ્સને સ્ટેકમાંથી ઉઠાવીને Heap મેમરીમાં એક સ્પાસ પ્રાઇવેટ ઓબ્જેક્ટમાં લોક કરી દે છે, જેને [[Scopes]] Closure કહેવાય છે.

૬૨. Data Encapsulation (ડેટા કેપ્સ્યુલેશન પ્રોટોકોલ)

ઇમ્પોર્ટન્ટ વેરીએબલ્સને ફંક્શનની અંદર છુપાવીને (Private કરીને) બહારની દુનિયાના કોડથી પ્રોટેક્ટ કરવા. ડેટા બદલવા કે જોવા માટે આપણે માત્ર કસ્ટમ સુરક્ષિત ગેટ્સ (Getters and Setters) જ ઓપન રાખીએ છીએ.

૬૩. Factory Functions Design Pattern (ફેક્ટરી ફંક્શન્સ ડિઝાઇન પેટર્ન)

સોફ્ટવેર આર્કિટેક્ચરની એવી સુંદર પેટર્ન જેમાં કોઈ class કે new કીવર્ડ વગર, સાદા ફંક્શનની અંદર ક્લોઝર સેટ કરીને ઓટોમેટિક એક સિક્યોર કસ્ટમ ઓબ્જેક્ટ મેન્યુફેક્ચર (બનાવીને) બહાર મોકલવામાં આવે છે.

૬૪. Function Currying & Infinite Chains (ફંક્શન ચેઇનિંગ)

એકસાથે બધી આર્ગ્યુમેન્ટ્સ લેવાના બદલે ફંક્શનને એવા ટુકડાઓમાં ફેરવવું જ્યાં દરેક ફંક્શન માત્ર ૧ જ આર્ગ્યુમેન્ટ લે અને અંદરથી બીજું ફંક્શન પરફોર્મ કરે (દા.ત. `sum(10)(20)(30)()`).

૬૫. The 6 Event Loop Phases (ઇવેન્ટ લૂપના ૬ ચક્રો)

લૂપનો દરેક રાઉન્ડ (Tick) આ ૬ ફેઝમાંથી પસાર થાય છે: Timers (setTimeout) -> Pending OS Callbacks -> Idle/Prepare (સિસ્ટમ ઇન્ટરનલ) -> Poll (નવો નેટવર્ક ડેટા બ્લોકિંગ) -> Check (setImmediate) -> Close Callbacks (કનેક્શન ક્લીનઅપ).

૬૬. Microtask Queue Priority Hijacking (માઇક્રોટાસ્ક વીઆઇપી ક્યુ)

આ ઇવેન્ટ લૂપની સૌથી શક્તિશાળી VIP લાઇન છે જેમાં બધા Promises (.then/.catch) અને async/await ની કોડ ઊભા રહે છે. લૂપ ગમે તે ફેઝમાં હોય, પણ જ્યાં સુધી આ માઇક્રોટાસ્ક ક્યુ પૂરેપૂરી ખાલી ન થાય, ત્યાં સુધી તે આગળ વધી શકતું નથી.

૬૭. process.nextTick() Scheduling (સુપર કમાન્ડર)

Node.js નો આ સર્વોચ્ચ કમાન્ડર છે જેનો પાવર પ્રોમિસ કરતાં પણ પહેલો છે. ઇવેન્ટ લૂપ પોતાનો ચાલુ ફેઝ પૂરો કરીને આગલા સ્ટેપ પર પગ મૂકે તેની બરાબર વચ્ચે આ nextTick નો કોડ સૌથી પહેલા રન કરવો જ પડે છે.

૬૮. Garbage Collection Generational Hypothesis (મેમરી વિભાગીય વિભાજન)

V8 એન્જિન આખી હીપ મેમરીને બે ભાગમાં વહેંચે છે—New Space (Young Generation) જ્યાં ટૂંકા આયુષ્યવાળા નવા ઓબ્જેક્ટ્સ સેવ થાય છે અને Old Space (Old Generation) જ્યાં લાંબુ જીવતા કાયમી ઓબ્જેક્ટ્સ પ્રમોટ થાય છે.



PART 6: THE PROTOTYPE PIPELINE & OOP LAYOUTS (વારસો અને ક્લાસ)

૬૯. Prototypal Inheritance (પ્રોટોટાઇપલ વારસો)

જાવાસ્ક્રિપ્ટનો મૂળભૂત ઇન-બિલ્ટ નિયમ જેના દ્વારા એક સામાન્ય ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની બધી જ પ્રોપર્ટીઝ અને મેથડ્સને વગર કોઈ ક્લાસ ડાયરેક્ટ ઉધાર લઈને વાપરી શકે છે.

૭૦. Prototype Chain (પ્રોટોટાઇપ સાંકળ કનેક્શન)

જ્યારે કોઈ ઓબ્જેક્ટમાં કોઈ કી કે ફંક્શન ન મળે, ત્યારે V8 એન્જિન પાછળ છુપાયેલા લિંક વાયરો દ્વારા તેના પિતા અને દાદા (ગ્લોબલ Object.prototype) સુધી ઉપર ચેક કરતું જાય છે, જેને પ્રોટોટાઇપ ચેઇન કહેવાય છે.

૭૧. __proto__ Reference Pointer (ડન્ડર પ્રોટો વાયર)

દરેક સિંગલ ઓબ્જેક્ટની અંદર રહેલી એક છુપી પ્રોપર્ટી જે રેમ (RAM) ની અંદર પોતાના અસલી પેરેન્ટ ઓબ્જેક્ટના મેમરી સરનામાં સાથે સીધી કનેક્ટ (લિંક) થયેલી હોય છે.

૭૨. prototype Property (બ્લુપ્રિન્ટ પ્રોપર્ટી)

આ પ્રોપર્ટી સામાન્ય ઓબ્જેક્ટ પાસે નથી હોતી, તે માત્ર Constructor ફંક્શન્સ કે class પાસે જ હોય છે. આ એક એવો નકશો છે જેની અંદર મૂકેલું લોજિક લવિષ્યમાં બનતા બધા બાળકોને આપોઆપ વારસામાં મળે છે.

૭૩. JavaScript Polyfills (પોલીફીલ સપોર્ટ કોડ)

જૂના જમાનાના બ્રાઉઝર્સમાં આધુનિક જાવાસ્ક્રિપ્ટની કોઈ નવી મેથડ (જેમ કે .includes()) ચલાવવા માટે આપણે જાતે જે કસ્ટમ કોડ ટુકડો લખીને પ્રોટોટાઇપમાં ઇન્જેક્ટ કરીએ છીએ તેને પોલીફીલ કહેવાય છે.

૭૪. JavaScript Shims (શિમ પ્રોટોકોલ લાઇબ્રેરી)

પોલીફીલ જેવું જ મોટું કલેક્શન કે થર્ડ-પાર્ટી લાઇબ્રેરી જે જૂની સિસ્ટમ્સની ખામીઓ પકડીને રન-ટાઇમ પર જ આખો કોડનો રસ્તો એરર વગર બદલી નાખવાની ક્ષમતા ધરાવે છે.

૭૫. Syntactic Sugar Classes (ક્લાસ કીવર્ડનો સાચો ભ્રમ)

ES6 માં આવેલો class કીવર્ડ એ જાવાસ્ક્રિપ્ટમાં સાચો ક્લાસ નથી! તે માત્ર લખવા માટેની સુંદર રીત (Syntactic Sugar) છે, અંદરથી તો V8 એન્જિન જૂનું પ્રોટોટાઇપલ ઇનહેરિટેન્સ જ ચલાવે છે.

૭૬. Class super() Context Call (સુપર કન્ટેક્સ્ટ કોલ)

જ્યારે યાઇલ્ડ ક્લાસ પોતાના પેરેન્ટ ક્લાસના કન્સ્ટ્રક્ટર અથવા મેથડ્સને જાગ્રત કરવા માંગતો હોય, ત્યારે આ super() ફંક્શન કોલ કરવો કમ્પલસરી નિયમ છે.

૭૭. Private Class Fields (# હેશટેગ પ્રોપર્ટીઝ)

આધુનિક જાવાસ્ક્રિપ્ટમાં ક્લાસની અંદર પ્રોપર્ટીના નામની આગળ હેશટેગ (#) લગાવવાથી તે ક્લાસની બહાર ૧૦૦% અદ્રશ્ય (પ્રાઇવેટ લોક) થઈ જાય છે, તેને કોઈ મોડિફાય નથી કરી શકતું.

🔵 PART 7: FAULT HARDENING, PROXIES & STREAMS (સિક્યોરિટી અને કંટ્રોલ)

૭૮. "use strict"; (Strict Mode કડક ચોકીદાર)

ફાઇલની સૌથી ઉપર આ પ્રોટોકોલ લખતા જ એન્જિન હાઇ-સિક્યોરિટી મોડમાં સ્વયં થાય છે. તે નાની ભૂલો, ડેટા લીક, અને કીર્ડ વગર વેરીએબલ બનાવવાની જોખમી આદત પર તુરંત લાલ એરર આપે છે.

૭૯. Error-Handling Middleware Boundary (એરર આઇસોલેશન)

પ્રોગ્રામમાં ગમે ત્યાં આવતી અણધારી ભૂલોને કેચ કરવા માટે વપરાતું કવર, જે ચાલુ રન-ટાઇમે એપ્લિકેશનને અચાનક બંધ કે ક્રેશ થતાં અટકાવે છે.

૮૦. Two-Way Authenticated Cryptography (AES-256-GCM સિક્યોરિટી)

ડેટાબેઝ કે નેટવર્ક પર ડેટા મોકલતા પહેલા તેને લાઇવ મિલિટરી લેવલ કી અને એન્ક્રિપ્શન ટેગ (authTag) થી લોક કરવાની સર્વોચ્ચ સિક્યોરિટી આર્કિટેક્ચર સિસ્ટમ.

૮૧. High-Performance Readable Streams (રીડેબલ ચંક્સ ઇન્જેસ્ટર)

બહુ મોટી ફાઇલ (જેમ કે ૫ GB નો વીડિયો) ને એકસાથે રેમમાં લોડ કરવાના બદલે તેને નાના-નાના બાઇટ્સના ટુકડાઓમાં (Chunks) વહેંચીને સ્પીડમાં રીડ કરવાની મેથડ.

૮૨. Consumer Saturation Backpressure (બેકપ્રેશર કંટ્રોલ)

જ્યારે ડેટા રીડ કરવાની સ્પીડ બહુ ફાસ્ટ હોય અને ડેટા રાઇટ કરવાનું કમ્પ્યુટર સ્લો હોય, ત્યારે કન્ટેનર ભરાઈ ન જાય તે માટે ફાસ્ટ ડેટાને હવામાં થોડીવાર હોલ્ડ (રોકી) રાખવાની સ્માર્ટ સિસ્ટમ.

૮૩. ESM Static Tree Splitting & Tree Shaking

આધુનિક import/export સિસ્ટમ જેનાથી પ્રોજેક્ટ બિલ્ડ થતી વખતે કોડનો જે ભાગ (ફંક્શન) ક્યાંય વપરાયો ન હોય, તેને પ્રોડક્શન બંડલમાંથી આપોઆપ કાચમ માટે સાફ કરી નાખવામાં આવે છે.

🌐 PART 8: BROWSER ENVIRONMENT & DESKTOP STORAGE (બ્રાઉઝર સ્ટોરેજ)

૮૪. DOM Reflow & Repaint Overhead (પેજ પર્ફોર્મન્સ લોડ)

સાદા જાવાસ્ક્રિપ્ટથી જ્યારે આપણે પેજના અક્ષરો કે સાઇઝ બદલીએ, ત્યારે બ્રાઉઝરે આખી વેબસાઇટના લુક અને કલરની ફરી ગણતરી કરવી પડે છે, જે બ્રાઉઝરને ખૂબ હેંગ કરે છે.

૮૫. Event Capturing vs Bubbling (ઇવેન્ટ વહેવાની પ્રક્રિયા)

બટન ક્લિક થતાં ઇવેન્ટ પહેલા ઉપરથી નીચે ચાઇલ્ડ તરફ જાય છે જેને Capturing કહેવાય છે, અને બટન દબાયા પછી ઇવેન્ટ પાછી નીચેથી ઉપર પેરેન્ટ તરફ વહે છે જેને Bubbling કહેવાય છે.

૮૬. localStorage Engine (લોકલ સ્ટોરેજ કાયમી તિજોરી)

બ્રાઉઝરની અંદર આપેલી 5 MB ની એવી કાયમી કી-વેલ્યુ સ્પેસ, જેમાં સાચવેલો ડેટા બ્રાઉઝર બંધ કરવા કે કમ્પ્યુટર બંધ કરવા છતાં વર્ષો સુધી સેવ જ રહે છે.

૮૭. sessionStorage Engine (સેશન સ્ટોરેજ ટેમ્પરરી તિજોરી)

આ પણ 5 MB ની જગ્યા આપે છે, પણ આની વેલિડિટી માત્ર એ સિંગલ બ્રાઉઝર ટેબ પૂરતી જ હોય છે. જેવું યૂઝર એ ટેબ બંધ કરી દેશે, એટલે આખો ડેટા રેમમાંથી આપોઆપ નાશ પામશે.

૮૮. HttpOnly Document Cookies Headers (સર્વર કુકીઝ લોક)

4 KB નું નાનું સ્ટોરેજ ડેટાબેઝ જે દરેક HTTP API Request ની સાથે આપોઆપ બેકએન્ડ સર્વર પર જાય છે. HttpOnly ટેગ લગાવવાથી બ્રાઉઝરનો કોઈ પણ જાવાસ્ક્રિપ્ટ કોડ તેને અડી શકતો નથી.

૮૯. IndexedDB Transactional Architecture (બ્રાઉઝર નોએસક્યુએલ ડેટાબેઝ)

બ્રાઉઝરની અંદર આપેલો એક આખો કમ્પીટ અસિન્ક્રોનસ NoSQL Database. તે હાર્ડડિસ્કના આધારે GB સુધીનો મોટો ઓબ્જેક્ટ ડેટા સાચવવાની છૂટ આપે છે, જેથી આખી એપ્લિકેશન ઓફલાઇન ચાલી શકે છે.

૯૦. MutationObserver Engine (લાઈવ DOM મોનિટરિંગ ચોકીદાર)

બ્રાઉઝરના HTML Tree ની અંદર રન-ટાઇમે થતા નાનામાં નાના ફેરફાર અથવા હેકરના ખોટા વાયરસ ટેગ ઇન્જેક્શન પર ચોકીદારની જેમ લાઈવ નજર રાખીને સેકન્ડમાં એલર્ટ ફાયર કરવાનું અલ્ટીમેટ એન્જિન.

PART 3: ADVANCED DATA TYPES MATRIX (એડવાન્સ ડેટા મેટ્રિક્સ)

૪૧. Array .map() Transformation Engine (એરે બદલવાનું મશીન)

ઓરિજિનલ એરેના ડેટાને ટય કર્યા વગર (Immutability), તેની એક-એક આઇટમ પર કસ્ટમ ગણતરી કે ફેરફાર કરીને મેમરીમાં તદ્દન નવો જ એરે ઓબ્જેક્ટ રિટર્ન કરવાનું હાઇ-ઓર્ડર એન્જિન.

૪૨. Array .filter() Selection Filter (એરે ફિલ્ટર ગાળણી)

એરેની દરેક આઇટમ પર શરત (Condition) ચેક કરે છે, જે આઇટમ શરત સાચી (true) પાડશે તેને નવા એરેમાં રાખીને બાકીના ડેટાને લિસ્ટમાંથી આપોઆપ ગાળીને (Delete કરીને) અલગ કાઢવાની મેથડ.

૪૩. Array .reduce() Aggregator Engine (એરે ભેગો કરવાનો ડબ્બો)

આખા એરેની બધી આઇટમ્સ પર લૂપ ચલાવીને, તેના ડેટાને ભેગો (Aggregate) કરીને છેલ્લે માત્ર એક સિંગલ વેલ્યુ (જેમ કે ઇ-કોમર્સ કાર્ટનું આખું ફાઇનલ બિલ) ગણીને આપતું Accumulator મશીન.

૪૪. Object & Array Destructuring (ડેટા બહાર કાઢવાની શોર્ટકટ રીત)

કોઈ મોટા ઓબ્જેક્ટ કે એરેની અંદરથી કોઈ સ્પેસિફિક ડેટાને વારંવાર લાંબી લાઇન લખ્યા વગર એક જ સેકન્ડમાં ડાયરેક્ટ બહાર કાઢીને નવા સ્વતંત્ર વેરીએબલ્સમાં લોક (Unpack) કરવાનો એડવાન્સ ES6 નિયમ.

૪૫. Spread Operator ... (મેમરી ડુપ્લિકેશન કીંગ)

જ્યારે ત્રણ ટપકાં એરે કે ઓબ્જેક્ટની આગળ મુકાય, ત્યારે તે તેનો જૂનો કૌંસ તોડીને બધો ડેટા બહાર ફેલાવી દે છે, જેનાથી મેમરી લિક તૂટે છે અને એક સેફ નવી મેમરી કૉપી (Shallow Copy) બને છે.

૪૬. Rest Operator ... (છુટોછવાયો ડેટા પેક કરવાનું મશીન)

Spread થી તદ્દન ઉલટું, આ ફંક્શનના પેરામીટર્સમાં કે ડિસ્ટ્રીબ્યુટિંગ કરતી વખતે ડાબી બાજુ વપરાય છે અને બાકી બચેલા તમામે તમામ ડેટાને ભેગો કરીને એક સિંગલ એરે (Array) માં પેક કરી દે છે.

૪૭. Template Literals & Interpolation (બેકટિક્સનો જાદુ)

Backticks (`) નો ઉપયોગ કરીને સ્ટ્રિંગની વચમાં જ કોઈ પણ ખસ ચિહ્ન વગર ડાયરેક્ટ ``${variable}`` લખીને ડાયનેમિક લાઈવ જાવાસ્ક્રિપ્ટ કોડ કે વેરીએબલ ઘુસાડવાનો મોડર્ન પ્રોટોકોલ.

૪૮. Optional Chaining ?. (સર્વર કેશ પ્રોટેક્શન ગેટ)

બ્રેકેટ્સ ઓબ્જેક્ટમાંથી ઊંડો ડેટા રીડ કરતી વખતે જો વચમાં કોઈ ફિલ્ડ ગાયબ (undefined કે null) હશે, તો આ ચોકીદાર કોડને કેશ થવા દેવાના બદલે શાંતિથી ત્યાં જ અટકીને સેફ undefined જવાબ આપી દેશે.

૪૯. Nullish Coalescing ?? (ડિફોલ્ટ વેલ્યુ સિક્યોરિટી ચોકીદાર)

જૂના OR (||) ઓપરેટરનો બગ ફિક્સર. આ માત્ર અને માત્ર null અને undefined ને જ ખોટું માને છે. જો યુઝરે કિંમત જાણીજોઈને ૦ (ઝીરો) કે ખાલી સ્ટ્રિંગ "" આપી હશે, તો આ તેને સાચી (Valid) ગણશે.

૫૦. JSON Methods (parse() & stringify())

stringify() તમારા જાવાસ્ક્રિપ્ટ ઓબ્જેક્ટને ઇન્ટરનેટ નેટવર્ક પર મોકલવા માટે JSON લખાણ (String) માં ફેરવે છે. parse() એ ઓનલાઇન આવેલા કાચા JSON લખાણને પાછા અસલી જાવાસ્ક્રિપ્ટ ઓબ્જેક્ટમાં કન્વર્ટ કરે છે.

PART 4: ALGORITHMIC COLLECTIONS & ADVANCED FUNCTIONS (એડવાન્સ ફંક્શન અને કલેક્શન)

૫૧. Primitive vs Composite Collections (કલેક્શનનો તફાવત)

Primitives માત્ર સિંગલ વેલ્યુ સાચવે છે, જ્યારે Composite Collections (જેમ કે સાદા ઓબ્જેક્ટ્સ અને એરે) મેમરીની અંદર એક આખો મોટો કસ્ટમ ડેટાબેઝ સ્ટ્રક્ચર ગ્રાફ હોલ્ડ કરવાની ક્ષમતા ધરાવે છે.

૫૨. Map Data Structure (મેપ કલેક્શન — એડવાન્સ ડિક્શનરી)

સાદા ઓબ્જેક્ટ જેવું જ કી-વેલ્યુ પેર કલેક્શન, પણ આનો પાવર એ છે કે આમાં તમે કી (Key) તરીકે માત્ર સ્ટ્રિંગ નહીં, પણ કોઈ બીજો આખો ઓબ્જેક્ટ, એરે કે ફંક્શનને પણ કી બનાવી શકો છો.

૫૩. Set Data Structure (સેટ કલેક્શન — યુનિક ડેટા બાસ્કેટ)

એક એવું ખાસ એરે જેવું લિસ્ટ ડેટા સ્ટ્રક્ચર જે પોતાના એરિયામાં ક્યારેય પણ કોઈ ડુપ્લિકેટ (એક સરખી) આઇટમ્સને ઘુસવા જ નથી દેતું, તે હંમેશા ફક્ત ૧૦૦% યુનિક વેલ્યુઝ જ સાચવે છે.

૫૪. WeakMap Architecture (વીક-મેપ — મેમરી સેફ ડિક્શનરી)

એક સ્પેશિયલ મેપ જેમાં કી (Key) તરીકે ફરજિયાત માત્ર ઓબ્જેક્ટ જ મૂકી શકાય. તે ઓબ્જેક્ટનો એકદમ પાતળો વાયર સાચવે છે, જેથી જો આખા પ્રોજેક્ટમાંથી એ ઓબ્જેક્ટ ડીલીટ થાય, તો આ તિજોરી તેને પકડી નથી રાખતી, Garbage Collector તેને તુરંત સાફ કરી નાખે છે.

૫૫. WeakSet Architecture (વીક-સેટ — મેમરી સેફ બાસ્કેટ)

WeakMap જેવું જ કલેક્શન જેમાં માત્ર ઓબ્જેક્ટ્સ સ્ટોર થાય છે. તે ઓબ્જેક્ટ્સના નકામા મેમરી રેફરન્સ પકડી રાખવાના બદલે તેને આપોઆપ ગાર્બેજ કલેક્શન (RAM Cleanup) થવા માટે મુક્ત રાખે છે.

૫૬. Higher-Order Functions Ingestion (હાઇ-ઓર્ડર ફંક્શન્સ ઇનપુટ)

જાવાસ્ક્રિપ્ટ એન્જિનનો એવો નિયમ જે ફંક્શનને એક ડેટાની જેમ ટ્રીટ કરે છે. આ નિયમ હેઠળ કોઈ ફંક્શન પોતાના ઇનપુટ કૌંસમાં બીજું આખું ફંક્શન સ્વીકારી શકે છે અથવા રિટર્નમાં નવું ફંક્શન આપી શકે છે.

૫૭. Callbacks Inversion of Control (કોલબેક કન્ટ્રોલ લોસ બગ)

જ્યારે આપણે આપણું કોઈ મહત્વનું કોલબેક ફંક્શન કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને સોંપી દઈએ, અને આપણને ખબર ન હોય કે તે આપણા ફંક્શનને ક્યારે, કેટલી વાર અને સાચું રન કરશે કે નહીં, તે કન્ટ્રોલ ગુમાવવાના બગને ઇન્વર્ઝન ઓફ કન્ટ્રોલ કહેવાય છે.

૫૮. Callback Hell Pyramid (પિરામિડ ઓફ ડ્રૂમ બગ)

જ્યારે એકપછી એક મલ્ટિપલ અસિન્ક્રોનસ કામો લાઈનસર કરવા માટે એક કોલબેકની અંદર બીજો, તેની અંદર ત્રીજો કોડ લખવો પડે, ત્યારે કોડ બહુ ગૂંચવાઈને વાંકોચૂંકો પિરામિડ બની જાય છે જેને ઉકેલવા Promises લાવવામાં આવ્યા.

૫૯. Function Custom Getters (ઇન્ટેલિજન્ટ રીડર ગેટ)

ઓબ્જેક્ટ કે ક્લાસની અંદર પ્રોપર્ટી રીડ કરવા માટે બનાવવામાં આવતું કસ્ટમ get ફંક્શન, જે બહારનો કોડ પ્રોપર્ટી કોલ કરે ત્યારે આપોઆપ બેકગ્રાઉન્ડમાં રન થઈને ડેટાને કસ્ટમાઇઝ કે મોડિફાઇ કરીને સ્ક્રીન પર મોકલે છે.

૬૦. Function Custom Setters (ઇન્ટેલિજન્ટ રાઇટર ગેટ)

ઓબ્જેક્ટ કે ક્લાસમાં નવો ડેટા લખવા કે સ્ટોર કરવા માટે બનાવવામાં આવતું કસ્ટમ set ફંક્શન, જે ડેટા અંદર સેવ થાય તેની બરાબર પહેલા રન થાય છે અને તેની સાઇઝ કે સિક્યોરિટી ચેક (Validation) કરીને જ મેમરી અપડેટ કરવા દે છે.

🌀 PART 3: ADVANCED RUNTIME CONCURRENCY (ઇવેન્ટ લૂપ અને અસિંક - ચાલુ)

૫૫. The Event Loop Mechanics (ઇવેન્ટ લૂપનું ચક્ર)

libuv C++ લાઇબ્રેરી દ્વારા ચાલતું એક એવું મેકેનિઝમ જે સિંગલ-થ્રેડેડ જાવાસ્ક્રિપ્ટને Non-blocking પાવર આપે છે. તે Call Stack ખાલી થતાં જ જાગે છે અને અસિન્ક્રોનસ કામોના કોલબેકસને લાઇનસર પ્રોસેસ કરે છે.

૫૬. The 6 Loop Phases (ઇવેન્ટ લૂપના ૬ ચક્રો)

લૂપનો દરેક રાઉન્ડ (Tick) આ ૬ ફેઝમાંથી પસાર થાય છે: Timers (setTimeout) -> Pending OS Callbacks -> Idle/Prepare (સિસ્ટમ ઇન્ટરનલ) -> Poll (નવો નેટવર્ક ડેટા બ્લોકિંગ) -> Check (setImmediate) -> Close Callbacks (કનેક્શન ક્લીનઅપ).

૫૭. Microtask Queue Priority Hijacking (માઇક્રોટાસ્ક વીઆઇપી ક્યુ)

આ ઇવેન્ટ લૂપની સૌથી શક્તિશાળી VIP લાઇન છે જેમાં બધા Promises (.then/.catch) અને async/await ની કોડ ઊભા રહે છે. લૂપ ગમે તે ફેઝમાં હોય, પણ જ્યાં સુધી આ માઇક્રોટાસ્ક ક્યુ પૂરેપૂરી ખાલી ન થાય, ત્યાં સુધી તે આગળ વધી શકતું નથી.

૫૮. process.nextTick() Scheduling (સુપર કમાન્ડર)

Node.js નો આ સર્વોચ્ચ કમાન્ડર છે જેનો પાવર પ્રોમિસ કરતાં પણ પહેલો છે. ઇવેન્ટ લૂપ પોતાનો ચાલુ ફેઝ પૂરો કરીને આગલા સ્ટેપ પર પગ મૂકે તેની બરાબર વચ્ચે આ nextTick નો કોડ સૌથી પહેલા રન કરવો જ પડે છે.

૫૯. Stack Overflow Vectors (મેક્સિમમ કોલ સ્ટેક કેશ)

જ્યારે કોઈ ફંક્શન પોતાને ને પોતાને જ કોઈ પણ અટકવાના નિયમ વગર વારંવાર કોલ કર્યા કરે (Infinite Recursion), ત્યારે Call Stack પૂરેપૂરો ઉપર સુધી ભરાઈ જાય છે અને એન્જિન Maximum call stack size exceeded ની એરર આપે છે.

૬૦. Async / Await Generator Pause Engine (અસિંક સ્ટેટ મશીન)

જ્યારે કોડમાં await આવે, ત્યારે V8 એન્જિન આખું સર્વર જામ નથી કરતું. તે બેકએન્ડમાં ફંક્શનના એક્ઝિક્યુશનને ત્યાં જ Pause (અટકાવી) દે છે. જવાબ આવતા જ માઇક્રોટાસ્ક ક્યુમાંથી કોડ પાછો ત્યાંથી જ Resume થાય છે.

⚙️ PART 4: COMPILATION, CLOSURES & OOP (આર્કિટેક્ચર નિયમો)

૬૧. [[Scopes]] Closure Memory Object (ક્લોઝર મેમરી ઓબ્જેક્ટ)

જ્યારે પેરેન્ટ ફંક્શન રન થઈને કોલ સ્ટેકમાંથી નાશ પામે, છતાં યાદવ્હ ફંક્શન તેના વેરીએબલ્સ વાપરતું હોય, ત્યારે V8 એન્જિન એ વેરીએબલ્સને સ્ટેકમાંથી ઉઠાવીને Heap મેમરીમાં એક ખાસ પ્રાઇવેટ ઓબ્જેક્ટમાં લોક કરી દે છે, જેને [[Scopes]] Closure કહેવાય છે.

૬૨. Data Encapsulation (ડેટા કેપ્સ્યુલેશન)

ઇમ્પોર્ટન્ટ વેરીએબલ્સને ફંક્શનની અંદર છુપાવીને (Private કરીને) બહારની દુનિયાના કોડથી પ્રોટેક્ટ કરવા. ડેટા બદલવા કે જોવા માટે આપણે માત્ર કસ્ટમ સુરક્ષિત ગેટ્સ (Getters and Setters) જ ઓપન રાખીએ છીએ.

૬૩. Factory Functions Design Pattern (ફેક્ટરી ફંક્શન ડિઝાઇન)

સોફ્ટવેર આર્કિટેક્ચરની એવી સુંદર પેટર્ન જેમાં કોઈ class કે new કીવર્ડ વગર, સાદા ફંક્શનની અંદર ક્લોઝર સેટ કરીને ઓટોમેટિક એક સિક્યોર કસ્ટમ ઓબ્જેક્ટ મેન્યુફેક્ચર (બનાવીને) બહાર મોકલવામાં આવે છે.

૬૪. Function Currying & Infinite Chains (ફંક્શન ચેઇનિંગ)

એકસાથે બધી આર્ગ્યુમેન્ટ્સ લેવાના બદલે ફંક્શનને એવા ટુકડાઓમાં ફેરવવું જ્યાં દરેક ફંક્શન માત્ર ૧ જ આર્ગ્યુમેન્ટ લે અને અંદરથી બીજું ફંક્શન રન કરે (દા.ત. `sum(10)(20)(30)()`).

૬૫. Prototypal Inheritance & Chains (પ્રોટોટાઇપ વારસો)

જાવાસ્ક્રિપ્ટનો વારસો નિયમ. આમાં એક ઓબ્જેક્ટ પાસે પ્રોપર્ટી ન હોય, તો તે પોતાની પાછળ છુપાયેલી લિંક સાંકળ (Prototype Chain) દ્વારા તેના પિતા સમાન ઓબ્જેક્ટની અંદર જઈને એ ફંક્શન કે ડેટા શોધીને વાપરી શકે છે.

૬૬. __proto__ (Dunder Proto) vs prototype (લેઆઉટ તફાવત)

`__proto__` એ દરેક સિંગલ ઓબ્જેક્ટની અંદર રહેલો અસલી જીવતો લિંક વાયર છે જે પેરેન્ટ સાથે કનેક્ટ રાખે છે. `prototype` એ ફક્ત Constructor ફંક્શન્સ કે ક્લાસ પાસે રહેલો એક ખાલી નકશો (Blueprint) છે જેમાં નવો લોજિક કોડ સ્ટોર થાય છે.

૬૭. JavaScript Polyfills & Shims (પોલીફિલ સપોર્ટ કોડ)

જો કોઈ કસ્ટમરના જૂના બ્રાઉઝરમાં આધુનિક જાવાસ્ક્રિપ્ટની કોઈ નવી મેથડ (જેમ કે .includes()) ઇન-બિલ્ટ ખાલી ન હોય, તો બ્રાઉઝર કેશ ન થાય તે માટે આપણે જાતે ઝીરોથી એ ફંક્શન કોડ લખીને બ્રાઉઝરના પ્રોટોટાઇપમાં ઘુસાડી દઈએ છીએ, એ સપોર્ટ કોડને Polyfill કહેવાય છે.

૬૮. ESM Static Tree vs CommonJS (CJS) (મોડ્યુલ સિસ્ટમ)

CommonJS (require()) જૂની પદ્ધતિ છે જે Synchronous અને Dynamic છે. ESM (import/export) આધુનિક સ્ટાન્ડર્ડ છે જે Asynchronous અને Static છે, આના લીધે V8 એન્જિન કોડ રન કરતા પહેલા જ આખો નકશો સમજીને નકામો કોડ ઓટો-ડીલીટ (Tree Shaking) કરી શકે છે.

૬૯. "use strict"; (Strict Mode Engine)

ફાઇલની સૌથી ઉપર આ લાઇન લખતા જ V8 એન્જિન સુપર-સિક્યોર કડક મોડમાં આવી જાય છે. તે નાની ભૂલો, ડેટા લીક, અને Accidental Globals ને તુરંત જ લાલ એરર આપીને ઓટો-બ્લોક કરી દે છે.

PART 5: BROWSER ENVIRONMENT & CLIENT STORAGE (બ્રાઉઝર સ્ટોરેજ)

૭૦. DOM Reflow / Layout & Paint Overhead (બ્રાઉઝર લોડ ઓવરહેડ)

સાદા જાવાસ્ક્રિપ્ટથી જ્યારે આપણે પેજ બદલીએ, ત્યારે બ્રાઉઝરે આખી વેબસાઇટના લુક, સાઇઝ અને કલરની ફરીથી ગણતરી કરવી પડે છે, જેને Reflow અને Paint કહેવાય છે. આ બ્રાઉઝર માટે બહુ જ સ્વો પ્રોસેસ છે, આના જ બચાવ માટે રીએક્ટ પોતાનો Virtual DOM વાપરે છે.

૭૧. Event Capturing vs Bubbling (ઇવેન્ટ વહેવાની પ્રક્રિયા)

બટન ક્લિક થતાં ઇવેન્ટ પહેલા ઉપરથી નીચે યાઇલ્ડ તરફ જાય છે જેને Capturing કહેવાય છે, અને બટન દબાયા પછી ઇવેન્ટ પાછી નીચેથી ઉપર પેરેન્ટ તરફ વહે છે જેને Bubbling કહેવાય છે.

૭૨. localStorage Engine (લોકલ સ્ટોરેજ કાયમી તિજોરી)

બ્રાઉઝરની અંદર આપેલી 5 MB ની એવી કાયમી કી-વેલ્યુ સ્ટોરેજ જગ્યા, જેમાં સેવ થયેલો ડેટા બ્રાઉઝર બંધ કરવા કે કમ્પ્યુટર રીસ્ટાર્ટ કરવા છતાં ક્યારેય પણ ઓટોમેટિક ડીલીટ થતો નથી.

૭૩. sessionStorage Engine (સેશન સ્ટોરેજ ટેમ્પરરી તિજોરી)

આ પણ 5 MB ની જગ્યા આપે છે, પણ આની વેલિડિટી માત્ર એ સિંગલ બ્રાઉઝર ટેબ (Session) પૂરતી જ હોય છે. જેવું યુઝર એ ટેબ બંધ કરી દેશે, એટલે આખો ડેટા ઓટોમેટિક રેમમાંથી સાફ થઈ જશે.

૭૪. Document Cookies (HttpOnly Headers લોક)

4 KB ની બહુ જ નાની સ્ટોરેજ સિસ્ટમ. આની ખાસિયત એ છે કે બ્રાઉઝર જ્યારે પણ સર્વર પર કોઈ નવી API રિક્વેસ્ટ મોકલે, ત્યારે કુકીઝનો ડેટા આપોઆપ હેડર સાથે બેકગ્રાઉન્ડ સર્વર પર વહેતો જાય છે. HttpOnly લોક લગાવવાથી જાવાસ્ક્રિપ્ટનો કોઈ કોડ તેને ક્યારેય બ્રાઉઝરમાંથી ચોરી શકતો નથી.

૭૫. IndexedDB Transactional Architecture (બ્રાઉઝર નોએસક્યુએલ ડેટાબેઝ)

બ્રાઉઝરની અંદર આપેલો એક આખો કમ્પીટ, અત્યંત પાવરફુલ Transactional NoSQL Database. તે ૧૦૦% Asynchronous (Non-blocking) છે અને હાર્ડિસકના આધારે GB સુધીનો વિશાળ ડેટા સ્ટોર કરવાની છૂટ આપે છે, જેનાથી આખી વેબસાઇટ ઇન્ટરનેટ વગર (Offline PWA Mode) ચાલી શકે છે.

૭૬. MutationObserver Engine (લાઈવ DOM ચોકીદાર)

બ્રાઉઝરના અસલી DOM Tree ની અંદર થતા નાનામાં નાના ફેરફાર (જેમ કે કોઈ હેકરે રન-ટાઇમ પર નવો HTML ટેગ ઘુસાડ્યો કે ક્લાસ નેમ બદલ્યું), તેને વાઈવ ટ્રેક કરીને સેકન્ડમાં સેન્ટ્રલ બેકઅપ એલર્ટ ફાયર કરવાનું અદ્યતન ઇન-બિલ્ટ મોનિટરિંગ એન્જિન.

PART 6: ADDITIONAL RUNTIME SECURITY & SPECIALIZED TERMS (વધારાના સિકેટ પ્રોટોકોલ્સ)

૭૭. Object Property Descriptors (પ્રોપર્ટી સિક્યોરિટી લોક)

ઓબ્જેક્ટની દરેક કી ની પાછળ છુપાયેલા ૩ સિક્યોરિટી વેલ્યુ લોક—writable (કિંમત બદલવી કે નહીં), enumerable (લૂપમાં દેખાડવી કે હાઇડ રાખવી), અને configurable (ડીલીટ કરવા દેવી કે લોક રાખવી).

૭૮. Class super() Context Call (સુપર કન્ટેક્સ્ટ વાયર)

જ્યારે ચાઇલ્ડ ક્લાસ પોતાના પેરેન્ટ ક્લાસના કન્સ્ટ્રક્ટર અથવા મેથડ્સને જાગ્રત કરવા માંગતો હોય, ત્યારે આ super() ફંક્શન કોલ કરવો કમ્પલસરી એન્જિનિયરિંગ નિયમ છે.

૭૯. Private Class Fields (# હેશટેગ સિક્યોરિટી)

આધુનિક જાવાસ્ક્રિપ્ટમાં ક્લાસની અંદર પ્રોપર્ટીના નામની આગળ હેશટેગ (#) લગાવવાથી તે ક્લાસની બહાર ૧૦૦% અદ્રશ્ય (પ્રાઇવેટ લોક) થઈ જાય છે, તેને કોઈ બહારથી મોડિફાય નથી કરી શકતું.

૮૦. Two-Way Authenticated Cryptography (AES-256-GCM સુરક્ષા)

ડેટાબેઝ કે નેટવર્ક પર ડેટા મોકલતા પહેલા તેને વાઈવ મિલિટરી લેવલ કી અને એન્ક્રિપ્શન ટેગ (authTag) થી લોક કરવાની સર્વોચ્ચ પ્રોડક્શન સિક્યોરિટી આર્કિટેક્ચર સિસ્ટમ.

૮૧. High-Performance Readable Streams (ડેટા સ્ટ્રીમ પાઇપલાઇન)

બહુ મોટી ફાઇલ (જેમ કે ૫ GB નો વીડિયો) ને એકસાથે રેમમાં લોડ કરવાના બદલે તેને નાના-નાના બાઇટ્સના ટુકડાઓમાં (Chunks) વહેંચીને સ્પીડમાં રીડ કરવાની મેથડ.

૮૨. Consumer Saturation Backpressure (બેકપ્રેશર કંટ્રોલર)

જ્યારે ડેટા રીડ કરવાની સ્પીડ બહુ ફાસ્ટ હોય અને ડેટા રાઇટ કરવાનું કમ્પ્યુટર સ્લો હોય, ત્યારે કન્ટેનર ભરાઈ ન જાય તે માટે ફાસ્ટ ડેટાને હવામાં થોડીવાર હોલ્ડ (રોકી) રાખવાની સ્માર્ટ સિસ્ટમ.

૮૩. JavaScript Shims (શિમ પ્રોટોકોલ લાઇબ્રેરી)

પોલીફીલ જેવું જ મોટું કલેક્શન કે થર્ડ-પાર્ટી લાઇબ્રેરી જે જૂની સિસ્ટમ્સની ખામીઓ પકડીને રન-ટાઇમ પર જ આખો કોડનો રસ્તો એરર વગર બદલી નાખવાની ક્ષમતા ધરાવે છે.

૮૪. Higher-Order Functions Ingestion (હાઇ-ઓર્ડર ફંક્શન્સ ઇનપુટ)

જાવાસ્ક્રિપ્ટ એન્જિનનો એવો નિયમ જે ફંક્શનને એક ડેટાની જેમ ટ્રીટ કરે છે. આ નિયમ હેઠળ કોઈ ફંક્શન પોતાના ઇનપુટ કૌંસમાં બીજું આખું ફંક્શન સ્વીકારી શકે છે અથવા રિટર્નમાં નવું ફંક્શન આપી શકે છે.

૮૫. Callbacks Inversion of Control (કન્ટ્રોલ લોસ બગ)

જ્યારે આપણે આપણું કોઈ મહત્વનું કોલબેક ફંક્શન કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને સોંપી દઈએ, અને આપણને કન્ટ્રોલ ન હોય કે તે આપણા ફંક્શનને ક્યારે અને કેટલી વાર ખોટું રન કરશે, તે બગને ઇનવર્ઝન ઓફ કન્ટ્રોલ કહેવાય છે.

૮૬. Map Data Structure (મેપ કલેક્શન — એડવાન્સ ડિક્શનરી)

સાદા ઓબ્જેક્ટ જેવું જ કી-વેલ્યુ પેર કલેક્શન, પણ આનો પાવર એ છે કે આમાં તમે કી (Key) તરીકે માત્ર સ્ટ્રિંગ નહીં, પણ કોઈ બીજો આખો ઓબ્જેક્ટ, એરે કે ફંક્શનને પણ કી બનાવી શકો છો.

૮૭. Set Data Structure (સેટ કલેક્શન — યુનિક બાસ્કેટ)

એક એવું ખાસ એરે જેવું લિસ્ટ ડેટા સ્ટ્રક્ચર જે પોતાના એરિયામાં ક્યારેય પણ કોઈ ડુપ્લિકેટ (એક સરખી) આઇટમ્સને ઘુસવા જ નથી દેતું, તે હંમેશા ફક્ત ૧૦૦% યુનિક વેલ્યુઝ જ સાચવે છે.

૮૮. WeakMap Architecture (વીક-મેપ મેમરી સેફ્ટી)

એક સ્પેશિયલ મેપ જેમાં કી તરીકે ફરજિયાત માત્ર ઓબ્જેક્ટ જ મૂકી શકાય. તે ઓબ્જેક્ટનો એકદમ પાતળો વાયર સાચવે છે, જો આખા પ્રોજેક્ટમાંથી એ ઓબ્જેક્ટ ડીલીટ થાય, તો આ તિજોરી તેને પકડી નથી રાખતી આપોઆપ ક્લીનઅપ થવા દે છે.

૮૯. WeakSet Architecture (વીક-સેટ મેમરી સેફ્ટી)

WeakMap જેવું જ કલેક્શન જેમાં માત્ર ઓબ્જેક્ટ્સ સ્ટોર થાય છે. તે ઓબ્જેક્ટ્સના નકામા મેમરી રેફરન્સ પકડી રાખવાના બદલે તેને આપોઆપ ગાર્બેજ કલેક્શન (RAM Cleanup) થવા માટે મુક્ત રાખે છે.

૯૦. JavaScript Modules (CJS vs ESM — મોડ્યુલ આર્કિટેક્ચર)

require() (CommonJS) જૂની પદ્ધતિ છે જે સિંક્રોનસ છે. import/export (ESM) આધુનિક સ્ટાન્ડર્ડ છે જે અસિંક્રોનસ અને સ્ટેટિક છે, આના લીધે v8 એન્જિન કોડ રન કરતા પહેલા જ આખો નકશો સમજીને નકામો કોડ ઓટો-ડીલીટ (Tree Shaking) કરી શકે છે.

PART 3: ADVANCED DATA TYPES MATRIX (એડવાન્સ ડેટા મેટ્રિક્સ)

૪૧. Array .map() Transformation Engine (એરે બદલવાનું મશીન)

ઓરિજિનલ એરેના ડેટાને ટય કર્યા વગર (Immutability), તેની એક-એક આઇટમ પર કસ્ટમ ગણતરી કે ફેરફાર કરીને મેમરીમાં તદ્દન નવો જ એરે ઓબ્જેક્ટ રિટર્ન કરવાનું હાઇ-ઓર્ડર એન્જિન.

૪૨. Array .filter() Selection Filter (એરે ફિલ્ટર ગાળણી)

એરેની દરેક આઇટમ પર શરત (Condition) ચેક કરે છે, જે આઇટમ શરત સાચી (true) પાડશે તેને નવા એરેમાં રાખીને બાકીના ડેટાને લિસ્ટમાંથી આપોઆપ ગાળીને (Delete કરીને) અલગ કાઢવાની મેથડ.

૪૩. Array .reduce() Aggregator Engine (એરે ભેગો કરવાનો ડબ્બો)

આખા એરેની બધી આઇટમ્સ પર લૂપ ચલાવીને, તેના ડેટાને ભેગો (Aggregate) કરીને છેલ્લે માત્ર એક સિંગલ વેલ્યુ (જેમ કે ઇ-કોમર્સ કાર્ટનું આખું ફાઇનલ બિલ) ગણીને આપતું Accumulator મશીન.

૪૪. Object & Array Destructuring (ડેટા બહાર કાઢવાની શોર્ટકટ રીત)

કોઈ મોટા ઓબ્જેક્ટ કે એરેની અંદરથી કોઈ સ્પેસિફિક ડેટાને વારંવાર લાંબી લાઇન લખ્યા વગર એક જ સેકન્ડમાં ડાયરેક્ટ બહાર કાઢીને નવા સ્વતંત્ર વેરીએબલ્સમાં લોક (Unpack) કરવાનો એડવાન્સ ES6 નિયમ.

૪૫. Spread Operator ... (મેમરી ડુપ્લિકેશન કીંગ)

જ્યારે ત્રણ ટપકાં એરે કે ઓબ્જેક્ટની આગળ મુકાય, ત્યારે તે તેનો જૂનો કૌંસ તોડીને બધો ડેટા બહાર ફેલાવી દે છે, જેનાથી મેમરી લિક તૂટે છે અને એક સેફ નવી મેમરી કૉપી (Shallow Copy) બને છે.

૪. configured. Rest Operator ... (છુટોછવાયો ડેટા પેક કરવાનું મશીન)

Spread થી તદ્દન ઉલટું, આ ફંક્શનના પેરામીટર્સમાં કે ડિસ્ટ્રીબ્યુટિંગ કરતી વખતે ડાબી બાજુ વપરાય છે અને બાકી બચેલા તમામે તમામ ડેટાને ભેગો કરીને એક સિંગલ એરે (Array) માં પેક કરી દે છે.

૪૭. Template Literals & Interpolation (બેકટિક્સનો જાદુ)

Backticks (` ) નો ઉપયોગ કરીને સ્ટ્રિંગની વચમાં જ કોઈ પણ પ્લસ ચિહ્ન વગર ડાયરેક્ટ `${variable}` લખીને ડાયનેમિક લાઈવ જાવાસ્ક્રિપ્ટ કોડ કે વેરીએબલ ઘુસાડવાનો મોડર્ન પ્રોટોકોલ.

૪૮. Optional Chaining ?. (સર્વર કેશ પ્રોટેક્શન ગેટ)

બેકએન્ડ ઓબ્જેક્ટમાંથી ઊંડો ડેટા રીડ કરતી વખતે જો વચમાં કોઈ ફિલ્ડ ગાયબ (undefined કે null) હશે, તો આ ચોકીદાર કોડને કેશ થવા દેવાના બદલે શાંતિથી ત્યાં જ અટકીને સેફ undefined જવાબ આપી દેશે.

૪૯. Nullish Coalescing ?? (ડિફોલ્ટ વેલ્યુ સિક્યોરિટી ચોકીદાર)

જૂના OR (||) ઓપરેટરનો બગ ફિક્સર. આ માત્ર અને માત્ર null અને undefined ને જ ખોટું માને છે. જો યુઝરે કિંમત જાણીજોઈને 0 (ઝીરો) કે ખાલી સ્ટ્રિંગ "" આપી હશે, તો આ તેને સાચી (Valid) ગણશે.

૫૦. JSON Methods (parse() & stringify())

stringify() તમારા જાવાસ્ક્રિપ્ટ ઓબ્જેક્ટને ઇન્ટરનેટ નેટવર્ક પર મોકલવા માટે JSON લખાણ (String) માં ફેરવે છે. parse() એ ઓનલાઇન આવેલા કાચા JSON લખાણને પાછા અસલી જાવાસ્ક્રિપ્ટ ઓબ્જેક્ટમાં કન્વર્ટ કરે છે.

PART 4: ALGORITHMIC COLLECTIONS & ADVANCED FUNCTIONS (એડવાન્સ ફંક્શન અને કલેક્શન)

૫૧. Primitive vs Composite Collections (કલેક્શનનો તફાવત)

Primitives માત્ર સિંગલ વેલ્યુ સાચવે છે, જ્યારે Composite Collections (જેમ કે સાદા ઓબ્જેક્ટ્સ અને એરે) મેમરીની અંદર એક આખો મોટો કસ્ટમ ડેટાબેઝ સ્ટ્રક્ચર ગ્રાફ હોલ્ડ કરવાની ક્ષમતા ધરાવે છે.

૫૨. Map Data Structure (મેપ કલેક્શન — એડવાન્સ ડિક્શનરી)

સાદા ઓબ્જેક્ટ જેવું જ કી-વેલ્યુ પેર કલેક્શન, પણ આનો પાવર એ છે કે આમાં તમે કી (Key) તરીકે માત્ર સ્ટ્રિંગ નહીં, પણ કોઈ બીજો આખો ઓબ્જેક્ટ, એરે કે ફંક્શનને પણ કી બનાવી શકો છો.

૫૩. Set Data Structure (સેટ કલેક્શન — યુનિક ડેટા બાસ્કેટ)

એક એવું ખાસ એરે જેવું લિસ્ટ ડેટા સ્ટ્રક્ચર જે પોતાના એરિયામાં ક્યારેય પણ કોઈ ડુપ્લિકેટ (એક સરખી) આઇટમ્સને ઘુસવા જ નથી દેતું, તે હંમેશા ફક્ત ૧૦૦% યુનિક વેલ્યુઝ જ સાચવે છે.

૫૪. WeakMap Architecture (વીક-મેપ — મેમરી સેફ ડિક્શનરી)

એક સ્પેશિયલ મેપ જેમાં કી (Key) તરીકે ફરજિયાત માત્ર ઓબ્જેક્ટ જ મૂકી શકાય. તે ઓબ્જેક્ટનો એકદમ પાતળો વાયર સાચવે છે, જો આખા પ્રોજેક્ટમાંથી એ ઓબ્જેક્ટ ડીલીટ થાય, તો આ તિજોરી તેને પકડી નથી રાખતી, Garbage Collector તેને તુરંત સાફ કરી નાખે છે.

૫૫. WeakSet Architecture (વીક-સેટ — મેમરી સેફ બાસ્કેટ)

WeakMap જેવું જ કલેક્શન જેમાં માત્ર ઓબ્જેક્ટ્સ સ્ટોર થાય છે. તે ઓબ્જેક્ટ્સના નકામા મેમરી રેફરન્સ પકડી રાખવાના બદલે તેને આપોઆપ ગાર્બેજ કલેક્શન (RAM Cleanup) થવા માટે મુક્ત રાખે છે.

૫૬. Higher-Order Functions Ingestion (ઘાઇ-ઓર્ડર ફંક્શન્સ ઇનપુટ)

જાવાસ્ક્રિપ્ટ એન્જિનનો એવો નિયમ જે ફંક્શનને એક ડેટાની જેમ ટ્રીટ કરે છે. આ નિયમ હેઠળ કોઈ ફંક્શન પોતાના ઇનપુટ કૌંસમાં બીજું આખું ફંક્શન સ્વીકારી શકે છે અથવા રિટર્નમાં નવું ફંક્શન આપી શકે છે.

૫૭. Callbacks Inversion of Control (કોલબેક કન્ટ્રોલ લોસ બગ)

જ્યારે આપણે આપણું કોઈ મહત્વનું કોલબેક ફંક્શન કોઈ થર્ડ-પાર્ટી લાઇબ્રેરીને સોંપી દઈએ, અને આપણને ખબર ન હોય કે તે આપણા ફંક્શનને ક્યારે, કેટલી વાર અને સાચું રન કરશે કે નહીં, તે કન્ટ્રોલ ગુમાવવાના બગને ઇન્વર્ઝન ઓફ કન્ટ્રોલ કહેવાય છે.

૫૮. Callback Hell Pyramid (પિરામિડ ઓફ હૂમ બગ)

જ્યારે એકપછી એક મલ્ટિપલ અસિન્ક્રોનસ કામો લાઈનસર કરવા માટે એક કોલબેકની અંદર બીજો, તેની અંદર ત્રીજો કોડ લખવો પડે, ત્યારે કોડ બહુ ગૂંચવાઈને વાંકોચૂંકો પિરામિડ બની જાય છે જેને ઉકેલવા Promises લાવવામાં આવ્યા.

૫૯. Function Custom Getters (ઇન્ટેલિજન્ટ રીડર ગેટ)

ઓબ્જેક્ટ કે ક્લાસની અંદર પ્રોપર્ટી રીડ કરવા માટે બનાવવામાં આવતું કસ્ટમ get ફંક્શન, જે બહારનો કોડ પ્રોપર્ટી કોલ કરે ત્યારે આપોઆપ બેકઅપમાં રન થઈને ડેટાને કસ્ટમાઇઝ કે મોડિફાય કરીને સ્ક્રીન પર મોકલે છે.

૬૦. Function Custom Setters (ઇન્ટેલિજન્ટ રાઇટર ગેટ)

ઓબ્જેક્ટ કે ક્લાસમાં નવો ડેટા લખવા કે સ્ટોર કરવા માટે બનાવવામાં આવતું કસ્ટમ set ફંક્શન, જે ડેટા અંદર સેવ થાય તેની બરાબર પહેલા રન થાય છે અને તેની સાઇઝ કે સિક્યોરિટી ચેક (Validation) કરીને જ મેમરી અપડેટ કરવા દે છે.
